SMART RESOURCE MANAGMENT AND NETWORK PRIORITIZATION

INDIVIDUAL THESIS

Sooriyage Suhansa kashmira

IT22059772

Individual component: Smart Resource Prediction and Network Prioritization

Group ID: R26-145

Report submitted in partial fulfilment of the requirements for the

BSc (Hons) in Information Technology Specializing in

Information Technology

Department of

Information Technology

Sri Lanka Institute of Information Technology

Sri Lanka

October 2026

DECLARATION AND SUPERVISOR CERTIFICATION

Candidate declaration. I declare that this thesis is my own work and does not incorporate, without acknowledgement, material previously submitted for a degree or diploma at any university or institute of higher learning. To the best of my knowledge, it contains no material previously published or written by another person except where acknowledgement is made in the text. I grant the Sri Lanka Institute of Information Technology the non-exclusive right to reproduce and distribute this thesis, in whole or in part, in print, electronic, or other media, while retaining the right to use the content in future work.

Academic integrity and AI disclosure. I have disclosed all use of generative AI and other automated tools in accordance with the module policy. I remain responsible for the accuracy, originality, citations, code, data, analyses, and conclusions in this thesis.

Student name:

Sooriyage Suhansa Kashmira

Student ID: IT22059772

Signature:  Date: 10/11/2026

Supervisor certification. The above candidate carried out this research for the bachelor's degree thesis under my supervision.

Supervisor: Prof. Anuradha Jayakody

Signature: _____ Date: _____

Co-supervisor, if applicable: Ms. Shashika Lokuliyana

Abstract

Relief-camp planning depends on changing population information, compatible stock records, and timely communication. This thesis examines the Smart Resource Prediction and Network Prioritization component of the SDRC-Box prototype through source inspection, reproducible artefact evaluation, and selected software tests. The resource path combines damped Theil–Sen population forecasting, explicit cohort rates, survey-derived food models, commodity matching, and inventory depletion calculations. The network path uses measured link indicators, two random forests, application priorities, persistent queues, and cloud acknowledgments. On a regenerated evaluation trace, network-state argmax accuracy was 92.40%, below persistence at 93.57%. A saved threshold policy achieved 49.04% escalation recall with a 12.28% false-alarm rate. Learned QoS increased conditional simulated delivery from 82.50% to 90.08%, alongside 12.23% buffering; critical delivery remained 93.20% under both policies. The available household-derived regression reproduced MAE of 1.1950 kg per household per day and R^2 of 0.0894. Generated population back-tests favored the damped configuration at seven days, reducing MAE against flat hold by 33.2% in the normal scenario and 35.8% in the burst scenario. Selected cloud and Python checks passed, while eleven of eighty-eight local policy assertions failed. The findings support transparent planning and inspectable communication contracts, with mixed predictive results and incomplete integration. Independent camp demand, real-message outcomes, stakeholder validation, and individual ownership records remain unavailable. The contribution documented here is an evidence-based account and evaluation of the component; candidate-specific authorship requires confirmation.

Keywords: resource prediction, adaptive MQTT, population forecasting, disaster relief, inventory planning

ACKNOWLEDGEMENTS

I would like to express my sincere gratitude to my supervisor, **Prof. Anuradha Jayakody**, and my co-supervisor, **Ms. Shashika Lokuliyana**, for their invaluable guidance, constructive feedback, and continuous encouragement throughout this research. Their advice and expertise played a significant role in shaping the scope of this research component and establishing appropriate methods for validating its results.

I would also like to extend my appreciation to my group members, **Gallage Pasindu Hansana (IT22064868)**, who developed the *Smart Environmental Risk Monitoring and Location-Aware Flood Prediction* component, and **Shakya Nirmani Sisikelum Deerasinghe (IT22065780)**, who developed the *Psycho-Attentional Gated Education System* component. Their contributions and integration of their respective components were essential to the successful development of this research.

Furthermore, I sincerely thank all the participants who wore the ring and participated in testing the system. Their cooperation and feedback were valuable in evaluating the functionality and performance of the proposed solution.

Finally, I am grateful to the **Department of Information Technology at the Sri Lanka Institute of Information Technology (SLIIT)** for providing the facilities and academic environment necessary to carry out this research successfully.

TABLE OF CONTENTS

DECLARATION AND SUPERVISOR CERTIFICATION i

Abstract ii

ACKNOWLEDGEMENTS iii

TABLE OF CONTENTS iv

LIST OF FIGURES ix

LIST OF TABLES x

LIST OF ABBREVIATIONS xi

1 INTRODUCTION 1

1.1 Overall project context 1

1.2 Individual component context 2

1.3 Problem statement 3

1.4 Research gap 3

1.5 Aim and objectives 4

1.6 Research questions or hypotheses 4

1.7 Scope limitations and assumptions 5

1.8 Individual contribution 5

1.9 Thesis structure 6

2 LITERATURE REVIEW AND RELATED WORK 7

2.1 Review method and scope 7

2.2 Domain and theoretical foundations 8

2.2.1 Humanitarian planning and quantity definitions 8

2.2.2 MQTT delivery semantics and application reliability 9

2.2.3 Priority, capacity and network-layer differentiation 9

2.2.4 Predictive models and forecast baselines 10

2.3 Related research 11
2.3.1 Adaptive messaging studies 11
2.3.2 Forecast evaluation and temporal dependence 11
2.3.3 Dietary data and transfer to camp planning 12
2.4 Existing systems products and techniques 12
2.5 Gap synthesis and conceptual framework 13
3 RESEARCH METHODOLOGY 15
3.1 Research design 15
3.2 Data participants or study materials 16
3.2.1 Repository and study boundaries 16
3.2.2 Generated network trace 17
3.2.3 Household, child and displacement materials 18
3.3 Experimental or study procedure 19
3.3.1 Network rescoring and operating points 19
3.3.2 Food-model evaluation 19
3.3.3 Population rolling-origin evaluation 20
3.3.4 Software verification procedure 20
3.4 Measures and evaluation metrics 20
3.5 Data analysis 22
3.6 Ethics of privacy security and responsible Registration, identity and demographic information may be sensitive in a relief setting. The audit used source and derived research artefacts rather than exporting runtime personal records. Deployment would require an agreed access model, retention rules, consent, or another documented basis for processing, and restricted research access. The presence of TLS configuration or password fields does not establish that those controls are correctly deployed. No ethics approval number, security certification or consent procedure is invented. 22

3.7 Reproducibility and research artefacts 22
4 PRODUCT AND SOLUTION DESIGN 23
4.1 Requirements 23
4.2 Component architecture within the complete solution 24
4.3 Design alternatives and decisions 25
4.4 Detailed design 26
4.4.1 Network measurement and feature construction 26
4.4.2 Priority policy and queue behaviour 28
4.4.3 Population and demand calculation 29
4.4.4 Raw food, commodity mapping and stock depletion 31
4.5 Standards and professional practice 33
5 IMPLEMENTATION INTEGRATION AND TESTING 34
5.1 Development environment and technology stack 34
5.2 Component implementation 34
5.2.1 Training artefacts and runtime monitor 34
5.2.2 Adaptive transport and persistent synchronization 35
5.2.3 Population and child-intake services 36
5.2.4 Raw food, stock and inventory services 37
5.3 Integration with the other components 38
5.4 Verification and software testing 39
5.5 Deployment and operation 41
6 RESULTS AND VALIDATION 42
6.1 Research results 42
6.1.1 Network-state prediction 42
6.1.2 QoS classification and conditional delivery 43
6.1.3 Household-derived and child food estimates 45

6.1.4 Generated population forecasts 46
6.2 Product and technical results 48
6.3 User or stakeholder validation 48
6.4 Integration results 48
6.5 Summary of findings 49
7 DISCUSSION 50
7.1 Interpretation of findings 50
7.2 Comparison with literature baselines and existing solutions 51
7.3 Research contribution 51
7.4 Product contribution 52
7.5 Limitations and threats to validity 52
7.6 Implications 54
8 COMMERCIALISATION AND SUSTAINABILITY 55
8.1 Value proposition and target users 55
8.2 Market and competitive position 55
8.3 Business or sustainability model 55
8.4 Scalability intellectual property and compliance 56
8.5 Contribution to the Sustainable Development Goals 57
9 CONCLUSION AND FUTURE DIRECTIONS 58
9.1 Conclusion 58
9.2 Achievement of objectives 58
9.3 Individual research and product contributions 59
9.4 Future directions 59
REFERENCES 60
GLOSSARY 62
APPENDICES 63

Appendix A Research instruments or extended results 63

A.1 Extended network and population results 63

A.2 Reproduction commands and test interpretation 63

A.3 Selected API and persistence specification 64

A.4 Evidence identification and unavailable inputs 65

LIST OF FIGURES

Figure 1.1 Individual component within the SDRC-Box project 2
Figure 2.1 Conceptual framework linking planning, communication and evidence..... 14
Figure 3.1 Procedure used for source audit and artefact evaluation 15
Figure 3.2 Availability of input data and reproducible evaluation paths 17
Figure 4.1 Source-derived system architecture and principal protocols 24
Figure 4.2 Component dependencies and the separate legacy prediction adapter..... 25
Figure 4.3 Network measurement, prediction and effective-state workflow 27
Figure 4.4 Synchronisation sequence with transport completion and cloud application ACK 29
Figure 4.5 Population, cohort demand and commodity stock workflow..... 30
Figure 4.6 Logical persistent records for queues, synchronisation and inventory..... 32
Figure 5.1 Deployment definitions and operational service boundaries 40
Figure 6.1 Saved threshold trade-off: escalation recall versus false alarms 42
Figure 6.2 Conditional simulated delivery and buffering by message priority 43
Figure 6.3 Reproduced age-band coefficients for raw-food planning 44
Figure 6.4 Population MAE by horizon for two generated scenarios..... 46

LIST OF TABLES

Table 1.1 Research objectives and corresponding evidence 4
Table 2.1 Primary research and its relationship to the implemented component 7
Table 2.2 Capability comparison of techniques relevant to the prototype 12
Table 3.1 Study materials, provenance and permitted interpretation 16
Table 3.2 Evaluation measures and denominators 21
Table 4.1 Reconstructed requirements and available verification 23
Table 4.2 Current deterministic MQTT fallback policy 28
Table 4.3 Planning rates, units and evidential basis 30
Table 5.1 Implemented stack and verified execution context 33
Table 5.2 Principal network and resource integration contracts 37
Table 5.3 Selected software checks executed during the audit 39
Table 5.4 Current local policy-test disagreements and their interpretation 39
Table 6.1 Reproduced network-state results on the generated evaluation partition 41
Table 6.2 Reproduced priority-level QoS simulation results 42
Table 6.3 Reproduced raw-food regression coefficients and validation 44
Table 6.4 Saved child resampled-cohort MAPE and baseline comparison 45
Table 6.5 Seven-day generated population forecast results 45
Table 6.6 Research-question answers and current evidence boundary 48
Table 7.1 Principal validity threats and required corrective evidence 52
Table 8.1 Proposed operating model and budget evidence requirements 55
Table 9.1 Objective achievement within the available evidence 57
Table A.1 Saved argmax class metrics on the reproduced evaluation partition 62
Table A.2 Reproduced population MAE in people across generated horizons 62
Table A.3 Selected write, prediction-input and synchronisation contracts 63
Table A.4 Selected evidence-file identifiers 64

LIST OF ABBREVIATIONS

ACK — Acknowledgment
API — Application programming interface
AR — Autoregressive
CPU — Central processing unit
CSV — Comma-separated values
CV — Cross-validation
FAR — False-alarm rate
HTTP — Hypertext Transfer Protocol
IDMC — Internal Displacement Monitoring Centre
IoT — Internet of Things
JSON — JavaScript Object Notation
MAE — Mean absolute error
MAPE — Mean absolute percentage error
MQTT — Message Queuing Telemetry Transport
MQTT-SN — MQTT for Sensor Networks
OLS — Ordinary least squares
QoS — Quality of service
RAM — Random-access memory
RF — Random Forest
RTT — Round-trip time
SDG — Sustainable Development Goal
SDRC-Box — Smart Disaster Relief Camp Box
SLIIT — Sri Lanka Institute of Information Technology
SQL — Structured Query Language
SSE — Server-sent events
TCP — Transmission Control Protocol
TLS — Transport Layer Security
UUID — Universally unique identifier

1 INTRODUCTION

1.1 Overall project context

Disaster relief camps need decisions about people, supplies and communication to remain consistent as conditions change. A registration record can alter the expected number of meals; a stock issue can change the next replenishment date; and a delayed update can leave the cloud view behind the camp's local position. These dependencies make information management part of resource planning. Humanitarian logistics research describes coordination and uncertainty as central operational concerns, rather than treating relief delivery as a routine commercial supply chain [1]. This thesis studies how a software prototype can make some of those dependencies explicit and testable.

The surrounding project is identified in its repository as Smart Disaster Relief Camp Box, also abbreviated SDRC-Box. It includes sensor interfaces, local registration and distribution functions, cloud services, disaster-related processing, and education services. The individual component addressed here is Smart Resource Prediction and Network Prioritization. Its scope joins two concerns: forecasting food and drinking-water requirements from population information, and deciding how camp messages should be handled when the network deteriorates. The repository provides the basis for describing the implemented system [2]; the presence of a module does not, by itself, establish deployment or ownership by an individual researcher.

Figure 1.1 locates this component in the wider project. Sensor and fingerprint interfaces contribute operational information, while local applications record camp activities. Resource services depend principally on roster, population history and inventory data. A separate network monitor supplies state and quality-of-service decisions to the local messaging layer. The cloud platform receives synchronized records and produces planning outputs. This arrangement explains why an otherwise accurate demand calculation can still be operationally misleading if its inputs arrive late or represent incompatible units.

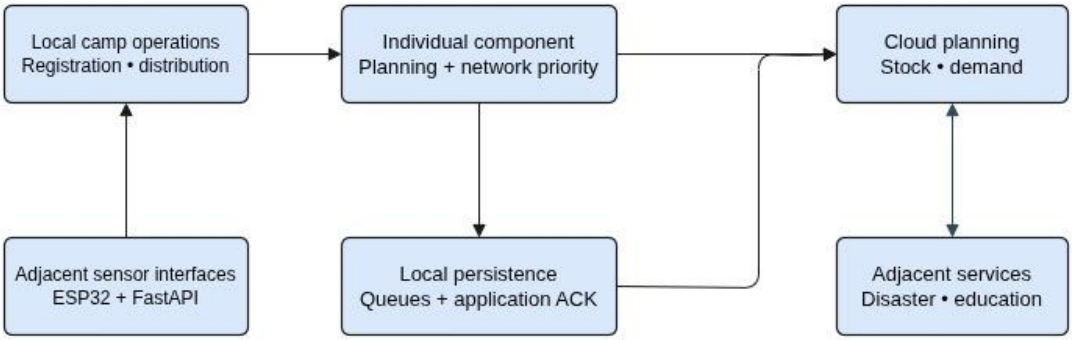


Figure 1.1 Individual component within the SDRC-Box project

1.2 Individual component context

The word resource has a specific meaning in this study: camp consumables, especially food and drinking water. The implemented forecasting path is not a model of future processor or memory demand. Although the repository includes device-status interfaces, those interfaces do not establish computational resource optimization. This distinction prevents the project title from implying a different research problem from the one supported by the source.

The resource component combines a population forecast with explicit cohort rates, survey-derived food coefficients, commodity matching, and stock depletion calculations. Population growth is estimated using a damped robust trend. Children can be assigned a saved dietary intake model when suitable roster records are available. Raw-food inventory uses a separate age-band model fitted to household-derived data. Drinking water demand follows a configured allowance. These mechanisms have different evidential bases and should be evaluated separately before their outputs are combined.

The network component measures latency, packet loss, jitter, broker connection timing, and queued work. Two saved random forests estimate future network state and a priority-dependent messaging action. Application logic then combines model output with current observations, policy freshness, recovery rules, and persistent queues. Message Queuing Telemetry Transport (MQTT) supplies the publish/subscribe transport and its defined delivery levels [3]. The project's prioritization takes place in application publishing and synchronization; it does not configure router packet scheduling.

1.3 Problem statement

A camp planning application needs more than a single daily consumption constant. Changes in population, age composition, attendance, and available stock can alter the quantity that should be prepared or replenished. Yet these inputs may be incomplete, and a recorded inventory quantity may use a different mass basis from a served-meal requirement. Treating all food quantities as interchangeable can produce a plausible number with an unclear operational meaning. The problem is therefore to provide an inspectable planning chain in which each input, conversion, and assumption can be traced.

Connectivity introduces a second difficulty. A uniform publication policy does not express the relative importance of registration, check-in, and routine distribution of updates. Increasing MQTT QoS for every message also does not create unlimited capacity or ensure that the destination database has accepted the record. Conversely, buffering work during poor conditions changes delivery time and can exhaust a finite queue. A useful policy must expose this trade-off instead of presenting higher conditional delivery as proof that every message arrived when needed.

The research problem is to assess whether the implemented combination of transparent demand modelling, application-level message prioritization, and persistent synchronization provides a defensible prototype for these tasks. Assessment must distinguish predictive behavior under the available data from complete product effectiveness. No independent camp-consumption study or complete measured network-delivery trial was supplied. The study consequently tests reproducible artefacts and software behavior while identifying the observations required for stronger operational conclusions.

1.4 Research gap

The literature includes humanitarian logistics, adaptive messaging, and established prediction methods. These strands offer useful foundations, but they do not automatically validate a camp application that couples out a roster-based demand

calculation to a local/cloud synchronization path. The gap addressed here is practical integration and evaluation: showing how population information becomes a resource requirement, how message importance becomes a transmission decision, and which parts of that chain are supported by current evidence.

The gap concerns practical integration and evaluation. Novelty of random forests, robust regression or MQTT is not claimed; Chapter 2 defines the limits of the focused review.

1.5 Aim and objectives

The aim is to develop an evidence-based account and evaluation of the implemented resource-planning and network-prioritization prototype, establishing its supported behavior and its limits. Table 1.1 translates this aim into objectives linked to verifiable artefacts. An objective is considered achieved only to the extent demonstrated by source inspection or an identified experiment; an implementation intention is not recorded as a measured outcome.

Table 1.1 Research objectives and corresponding evidence

Objective	**Operational definition**	**Evidence**
O1: Architecture	Identify the actual component, dependencies, and contracts.	Source audit; Chapters 4–5
O2: Network prediction	Compare future-state prediction with persistence and warning measures.	Generated-trace rescoring
O3: Messaging decisions	Evaluate QoS, buffering and completion boundaries.	Simulation and selected tests
O4: Resource planning	Explain and evaluate population, food, and stock calculations.	Derived-data fit; back-test; code
O5: Verification	Report tested behavior and unresolved integration evidence.	Audit logs and gap register

1.6 Research questions or hypotheses

Five research questions guide the evaluation. RQ1 asks how the saved network-state predictor compares persistence on the generated future-state task, including detection of deterioration. RQ2 asks how learned QoS and buffering compare with the generator's reference policy under their shared delivery assumptions. RQ3 asks how

damped population forecasting compares with flat and ordinary least-squares forecasts on generated rolling-origin histories. RQ4 asks whether the resource services provide a traceable calculation from cohort information to food, water and stock outlooks. RQ5 asks which persistence, priority, and inventory behaviors are supported by selected software checks.

The questions are exploratory. Comparisons report the effect, denominator and setting; a favorable isolated score is not assumed to establish operational benefit.

1.7 Scope limitations and assumptions

The technical scope covers the local network monitor, adaptive MQTT layer, persistent synchronization, cloud resource services, and inventory-related interfaces. Sensor and education subsystems are included only where they establish boundaries or supply data. Shelter and medical estimates in earlier notebooks are outside the evaluated component. The analyzed source is the working tree associated with base commit 9b8bfd0, including existing uncommitted inventory changes. Source conflicts and test disagreements are reported as part of that snapshot.

Resource planning assumes that registered population and recorded stock are usable operational inputs. The population projection holds cohort proportions fixed unless new information changes the input. Survey-derived coefficients are treated as planning estimates rather than nutritional prescriptions. The water service estimates a narrow allowance; it does not model the full water requirement of a camp. Standards define relevant distinctions, including survival intake and broader domestic use [4]. Missing operational inputs cannot be made reliable by assigning them a model-generated value.

Evaluation uses the available generated, derivative, and saved-artefact materials. Field, stakeholder, hardware, and commercial claims require evidence identified in the accompanying register.

1.8 Individual contribution

My individual contribution focused on integrating SmartCamp’s local Raspberry Pi environment with its cloud environment. I mapped the communication paths between the two and worked on the shared interfaces that support operations over unreliable

networks. I developed the network monitoring module and its integration with adaptive QoS, helping the system respond to degraded connectivity by prioritizing important messages and buffering others. I also worked on smart population demand forecasting to estimate camp resource needs and support resource management. The underlying algorithms, external data, and group-developed interfaces retain their original attribution.

The current audit supplies an evidence map, reproduced experiments, and reviewed test outcomes. It records preparation of this draft rather than undocumented historical development by the candidate; Appendix B discloses the assistance.

1.9 Thesis structure

Chapter 2 establishes conceptual and methodological foundations. Chapter 3 identifies the data, procedures, measures, and ethical boundaries of the evaluation. Chapter 4 explains the solution design, followed by implementation, integration, and testing in Chapter 5. Chapter 6 presents the verified results and unavailable validation of evidence. Chapter 7 interprets the findings and threats to validity. Chapter 8 considers a proposed sustainability model, and Chapter 9 concludes with priorities for further work. Appendices provide extended results, selected technical contracts, and academic integrity disclosure.

2 LITERATURE REVIEW AND RELATED WORK

2.1 Review method and scope

The review was organized around the implementation rather than a broad catalogue of disaster technologies. Five areas were examined: humanitarian supply planning, messaging under variable connectivity, short-horizon prediction, interpretable food-demand estimation, and reliable state synchronization. Searches used the names of implemented algorithms and combinations of MQTT, adaptive QoS, humanitarian logistics, and forecasting. Original papers, author-hosted copies, standards, official documentation and dataset catalogues were preferred. Publisher metadata was used when an original full text could not be accessed, with claims restricted to the material actually available.

This is a focused narrative review, without a preregistered search or exhaustive screening procedure. Selected sources establish relevant foundations and comparisons, rather than priority of invention. Each retained source supports a specific explanation in the thesis.

The comparison distinguishes the control location, predicted quantity, evaluation unit, and evidence setting of each approach. Those dimensions are particularly important here: publisher-side MQTT adaptation differs from broker-side delivery control; household food quantities differ from served camp meals, and simulator delivery probabilities differ from observed message outcomes. Table 2.1 sets out the main research connections. Results from other publications are not substituted for the project's own measurements.

Table 2.1 Primary research and its relationship to the implemented component

Source	Relevant method or setting	Relationship and boundary
Van Wassenhove [1]	Humanitarian logistics	Motivates coordination; no product demand evidence.
Palmese et al. [5]	Adaptive MQTT-SN; ns-3	Related adaptation; different protocols and evaluations.
Jeong et al. [6]	Broker-side QoS; equation-based analysis	Different control points; no shared trial.
Breiman [7]	Random forest ensemble	Algorithm foundation; project scores evaluated separately.

Source	Relevant method or setting	Relationship and boundary
NIST [8]; Gardner and McKenzie [9]	Robust slope and damped trend	Foundations for a separately capped implementation.
Hoerl and Kennard [10]	Penalized linear regression	Child estimator foundation; no camp transfer validation.
Hyndman and Athanasopoulos [11]	Rolling-origin validation	Supports horizon evaluation and chronological availability.

2.2 Domain and theoretical foundations

2.2.1 Humanitarian planning and quantity definitions

Van Wassenhove connects humanitarian logistics with preparedness, coordination and the conditions under which relief organizations operate [1]. For this component, the relevant implication is that information about requirements and available resources must be interpreted across actors. A calculated quantity is useful only if the person receiving it knows its period, unit, population basis, and assumptions. This thesis therefore treats transparent inputs, and evidence limits as part of the planning design.

The resource implementation contains three different quantity bases. Raw-food demand describes stock ingredients estimated from household quantities. Served food demand describes a meal-planning mass assembled from rates or child-intake predictions. Drinking water demand uses liters per person per day. These quantities answer related but different questions. Cooking changes to mass through added water and preparation, while a store may record dry rice or mixed commodities. A universal raw-to-cooked conversion factor would require evidence about the actual basket and preparation method; no such calibrated factor is established in the audited code.

The Sphere handbook separates survival intake from cooking and hygiene requirements. Its basic water table gives 2.5–3 liters per person per day for survival intake, with an additional 3–6 liters for basic cooking needs; its water-supply standard also discusses a minimum of 15 liters for broader domestic needs [4]. The cloud's 3-litre planning constant is therefore a narrow assumption, not a validated estimate of all camp water use. Standards guide for interpretation, but a standards-based constant is not a learned forecast or an observation of consumption.

2.2.2 MQTT delivery semantics and application reliability

MQTT separates publishing applications from subscribers through a broker. Its three QoS levels define distinct delivery exchanges: QoS 0 is best effort without a protocol acknowledgment, QoS 1 can deliver duplicates, and QoS 2 uses an additional exchange to control duplicate delivery within the protocol relationship [3]. These semantics concern the MQTT transport exchange. They do not certify that a cloud application has validated an envelope, committed an inventory transaction or updated a particular local record.

The source separates a transport callback, which can flush a buffered publication, from the application of ACK that completes a logical sync record. Receiver handling of repeated sync identifiers is therefore part of the reliability of contract.

The delay-tolerant networking architecture in RFC 4838 uses persistent store-and-forward operation to address disrupted connectivity [12]. The camp's implementation shares the practical idea of retaining work across a connection interruption, but it implements a local SQLite application queue over ordinary MQTT. It has no demonstrated bundle protocol, custody-transfer routing, or general delay-tolerant overlay. This comparison is useful as an architectural analogy, with the actual queue of contract providing the relevant implementation evidence.

2.2.3 Priority, capacity and network-layer differentiation

Message priority represents application importance; it is not the same variable as MQTT QoS. A registration update may be assigned of high importance even when the chosen transport level changes with current conditions. Buffering is another decision: it postpones transmission and can preserve work temporarily, but it consumes storage and introduces waiting time. The implementation uses four message priorities and a model action space that includes buffering. A meaningful evaluation must therefore measure how many messages are deferred as well as how transmitted messages are scored.

Differentiated Services describes an IP-layer architecture for traffic classification and per-hop treatment [13]. No corresponding packet marking or router enforcement was found in this component. Its term network prioritization refers to application

classification, publishing QoS selection, queue ordering, and synchronization scheduling. This boundary prevents application-level changes from being described as measured prioritization throughout the network. Such a claim would require configured intermediate devices and packet-level evidence.

2.2.4 Predictive models and forecast baselines

Breitman's random forest combines many randomized decision trees into an ensemble [7]. In this project, the classifiers use tabular network metrics and derived temporal features, with separate targets for future state and messaging action. The choice accommodates nonlinear interactions such as a rise in latency together with queue growth. It does not establish that this model is superior to a simple baseline on the available task. A persistence forecast remains necessary because the generator deliberately creates temporally persistent network conditions.

Theil–Sen trend estimation uses the median of slopes between pairs of observations [8]. Its relevance is a trend estimate that is less controlled by one extreme headcount than a least-squares line. The implemented service then modifies that estimate with a recent window, damping, anchoring, and caps. Gardner and McKenzie's work supplies a foundation for damped trend extrapolation [9]; the project's capped robust calculation is not an implementation of every element of their forecasting model. These distinctions matter when interpreting comparative errors.

For household food quantities, a linear combination of age-band counts has a direct planning interpretation. Ridge regression adds a penalty to linear estimation, changing the bias–variance trade-off when explanatory variables are correlated [10]. The saved child model uses this penalized approach with age, sex and province features. The household model instead uses unpenalized no-intercept regression. The two models should not be discussed as a single dietary predictor: their units, populations and estimands differ.

2.3 Related research

2.3.1 Adaptive messaging studies

Palmese, Redondi and Cesana study adaptive QoS for MQTT-SN and evaluate the approach in an ns-3 setting [5]. Their work supports the idea that transport handling can respond to changing conditions rather than remain fixed. The setting differs from the current system, which uses MQTT.js, a Python monitoring process and persistent application queues. MQTT-SN and MQTT have distinct protocol contexts, and the camp's predictor also attempts a future-state task. A published simulation gain cannot be transferred as a numerical expectation for this deployment.

Jeong, Park and Kyung describe broker-side network-aware MQTT delivery that selects a QoS level according to application latency and loss requirements. The publisher record reports equation-based performance analysis and identifies experimental verification as future work [6]. The camp component places its policy principally at the publisher and local synchronization layer. It uses constructed priority targets and permits deferral. Both approaches relate network conditions to application requirements, but their decision points and validation evidence differ.

2.3.2 Forecast evaluation and temporal dependence

Hyndman and Athanasopoulos describe time-series cross-validation through successive forecast origins, where each forecast uses only information available at that origin [11]. This principle supports the population's back-test and the need to report errors by horizon. A one-day model and a seven-day model can have very different error characteristics even when their forecasts come from the same equation. It also explains why an inventory forecast exposed at 30 or 90 days requires evidence beyond a seven-day evaluation.

Temporal dependence affects uncertainty as well as splitting. Several forecasts from one generated population history share observations and future outcomes. Their errors are therefore not independent samples merely because they occupy different rows. Resampling complete histories would preserve more of that dependence than resampling individual forecast errors. Efron's bootstrap provides a general resampling foundation [14], but the unit selected for resampling must still match the study

structure. No inferential interval based on an invented independent sample size is reported here.

2.3.3 Dietary data and transfer to camp planning

The FAO catalogue identifies the child source as Dietary Intakes of Sri Lankan Children 6–14 years, 2021, produced by Up field Research and Development BV [15]. The repository's training script and saved metadata report 653 retained subjects. The catalogue establishes the survey's origin and population; it does not independently confirm every transformation or validation score in the local m=odel's artefact. The original subject and consumption of CSVs were absent from the inspected inputs, limiting reproduction of that preparation path.

IFPRI identifies the BRIGHT Integrated Household Survey of Sri Lanka, 2024–2025 and its public data release [16]. The locally available derivative contains household-level food quantities and age-band counts. Reproducing a regression on that derivative provides useful evidence about the fitted relationship, but it does not recreate the original unit-conversion and filtering chain. Nor does a household survey automatically represent communal cooking in a relief camp. This application is a transfer of a planning relationship between settings, with that transfer requiring independent validation.

2.4 Existing systems products and techniques

The relevant alternatives are defined by capabilities rather than unverified product rankings. A spreadsheet or rule-based calculator can apply per-person rates transparently, but population updates and stock movements require an explicit recording process. A generic inventory ledger can preserve receipts and issues while leaving the forecast assumptions outside its scope. A fixed-QoS publisher can simplify operation, but it does not respond to message importance and predicted deterioration. Table 2.2 compares these approaches as design alternatives; it is not a commercial product trial.

Table 2.2 Capability comparison of techniques relevant to the prototype

Technique	**Supported capability**	**What remains outside it**
Fixed-rate calculator	Simple auditable population rate estimate	Composition, stock reconciliation and validated trend
Inventory ledger	Receipts, issues and balance history	Forecast assumptions and future population
Fixed-QoS publisher	Consistent transport configuration	Dynamic importance, deferral and freshness
Persistent application queue	Retains work across some interruptions	Unlimited capacity or deadline guarantee
Current prototype	Combines explicit demand and adaptive local handling	Field effectiveness and full integration verification

Several simpler techniques remain in credible baselines. Holding the current headcount is a population forecast with no estimated trend. Holding the current network state is a persistence classifier. A mean dietary intake rate can estimate a total without individual covariates. Their simplicity makes it easy to reproduce and difficult to misinterpret. A more complex model must justify its additional inputs and maintenance through a comparison that measures the operationally relevant outcome, rather than through its own score in isolation.

2.5 Gap synthesis and conceptual framework

Figure 2.1 summarizes the framework used throughout the thesis. Population and roster information enter an explicit demand calculation; inventory balances and commodity mappings translate that demand into an outlook. Network observations and message importance enter a separate policy calculation; persistent queues and acknowledgments determine how records become available in the cloud. Evaluation sits across these chains and asks whether each claim has the required input and outcome evidence.

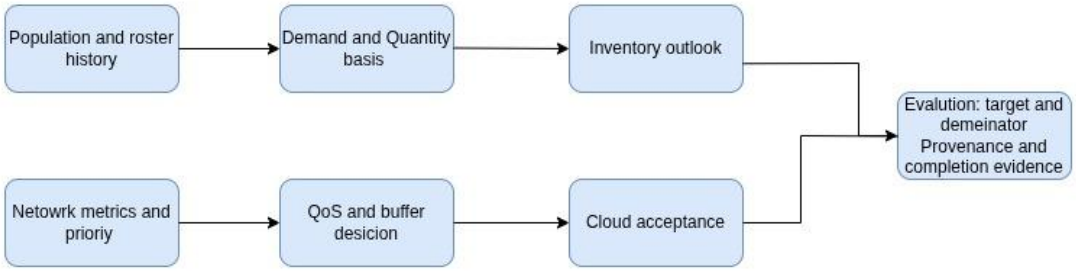


Figure 2.1 Conceptual framework linking planning, communication and evidence

The synthesis identifies three practical gaps. First, demand estimates need consistent units and a documented transfer from survey relationships to camp decisions. Second, network adaptation needs outcomes that include deferral, deadlines, and application completion. Third, integration needs explicit contracts and tests rather than topology alone. Addressing these gaps gives the prototype a research role even where its present results are mixed. The next chapter operationalizes this framework through separate data descriptions, reproducible procedures, and measures that retain their appropriate denominators.

3 RESEARCH METHODOLOGY

3.1 Research design

The study uses retrospective artefact evaluation to examine the supplied implementation and its available experimental evidence. Design-science research provides a relevant perspective because the object of study is an information-system artefact whose behavior can be evaluated [17]. This framing does not establish that every original development activity is followed by a documented design-science process. The confirmed procedure in this thesis is the present source audit, reconstruction of evaluation settings, reproduction where inputs permit, and comparison of supported results with the component's aims.

Three kinds of assessment are kept separate. Implementation inspection establishes what a function calculates or what an interface accepts. Model evaluation establishes error under a specified dataset and partition. Software testing establishes selected assertions under a defined fixture. None is interchangeable with stakeholder acceptance or operational effectiveness. Figure 3.1 shows the sequence from evidence of inventory to bounded interpretation. A result enters the thesis only when its source, procedure, and permissible interpretation can be identified.

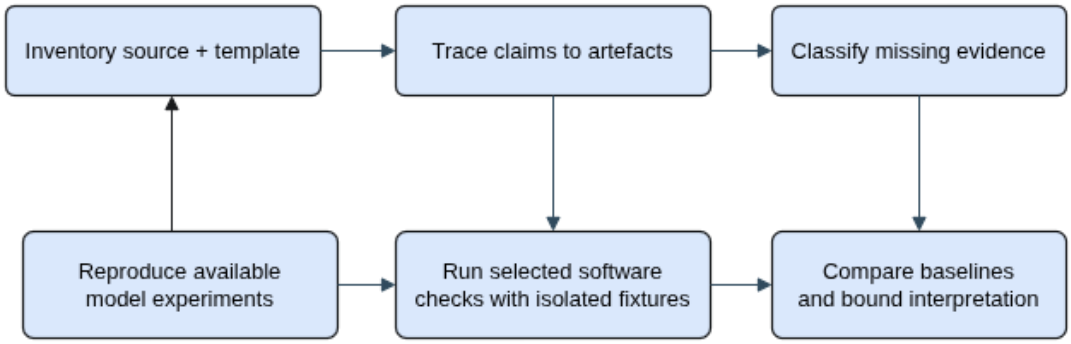


Figure 3.1 Procedure used for source audit and artefact evaluation

3.2 Data participants or study materials

3.2.1 Repository and study boundaries

The repository inventory covers 656 project files and indexes 508 source or document files, including notebook source and stored outputs. Dependencies, Git internals, build

products and environment directories were excluded from the project-content index. Credentials were excluded from research outputs. Runtime personal records were not exported as a study dataset. Manual review concentrated on forecasting, monitoring, messaging, synchronization and inventory; adjacent modules were inspected to identify their integration boundaries. This is broad repository coverage, not a claim of formal verification of every source line.

The analyzed working tree includes uncommitted inventory changes and unresolved interface conflicts. The evidence package records file hashes, routes and conflict locations. Table 3.1 identifies the materials, and Figure 3.2 distinguishes complete local evaluation paths from missing upstream inputs.

Table 3.1 Study materials, provenance and permitted interpretation

Material	**Availability and scale**	**Evidence boundary**
Network generator	39,987 usable rows; seed 42; saved models	Constructed states, actions and delivery probabilities
Household derivative	6,802 households; 27,689 people	Fit reproduced; original preparation inputs absent
Child model summary	653 reported subjects; ages 6–14	Saved results; raw input files absent
Displacement CSV	78 rows; 52 unique event IDs	Event scale for generated histories; export unresolved
Population histories	200 × 21 days per scenario	Generated normal and burst scenarios
Controlled-profile log	60 rows with applied network settings	No complete message outcomes
Source and test fixtures	Working tree; isolated temporary databases	Selected behaviour; no recruited participants

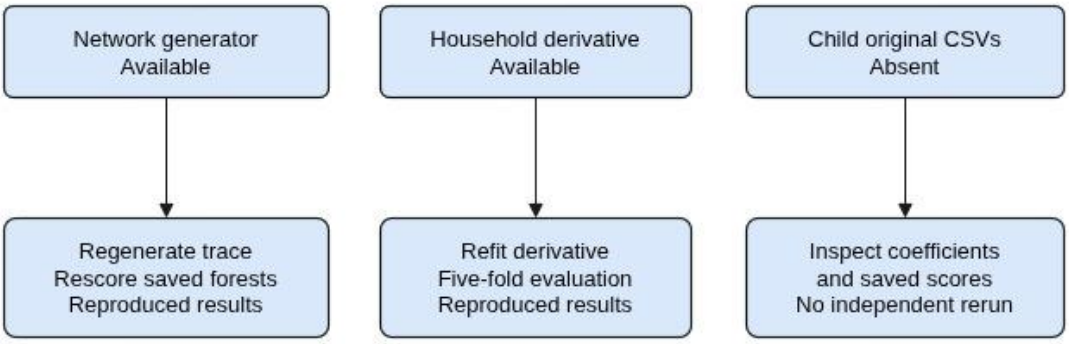


Figure 3.2 Availability of input data and reproducible evaluation paths

3.2.2 Generated network trace

The network generator creates a latent stress process with autoregressive momentum 0.93, stochastic changes, and recovery. It maps the process to round-trip time, loss, jitter, broker timing, bridge status, and queue variables. State labels are constructed from configured metric thresholds. A critical current state is assigned for loss of at least 15%, round-trip time of at least 1,000 ms or a failed bridge. Degraded conditions include loss of at least 5%, round-trip time of at least 300 ms or queue depth of at least 100. These definitions encode assumptions rather than discovered network-state categories.

The future target is the most severe state over the following ten samples. The nominal generated cadence is 30 seconds, giving a nominal five-minute look-ahead. For a request of 40,000 samples with seed 42, future-target construction leaves 39,991 generated rows. Removing four rolling-feature warm-up rows leaves 39,987 usable observations. The chronological 80/20 split contains 31,989 training and 7,998 evaluation observations. The deterministic generator was rerun in memory for rescoring; the missing original training CSV was not represented as an observed trace.

QoS labels use explicit delivery probabilities for healthy, degraded, and bad conditions at QoS 0, 1 and 2. The values are 0.70/0.95/0.99, 0.40/0.75/0.90 and 0.10/0.30/0.60 respectively, with zero probability when the bridge is unavailable. Required priority targets are 0.70, 0.90, 0.95, and 0.99. The generator selects the minimum sufficient

QoS or buffering, except for critical best-effort treatment when the target is unattainable. This label construction is part of the experiment and cannot be called measured delivery reliability.

3.2.3 Household, child and displacement materials

The available household derivative contains 6,802 rows, 27,689 counted people, nine province codes, and 25 district codes. No retained column null values were found. Its preparation script selects reported food records, applies survey unit factors, converts weekly quantities to daily mass and removes beverage-labelled items for the solid-food target. Household size and total quantity filters are also present. The original modules and conversion codebook were absent at the expected input location. The IFPRI release verifies the source survey [16], while the local CSV supports only the derivative and fitted-model reproduction.

The child artefact reports 653 subjects aged 6–14, with age, sex and province inputs. Its target sums 24-hour food quantities and expresses them in kilograms per child per day. The survey source is verified by the FAO catalogue [15]. Subject and consumption files were unavailable, so counts, preprocessing outcomes and child validation scores remain saved-artefact evidence. Weight and height were not collected by the model's documented input path and are not claimed as predictors. Age clamping at runtime does not extend the survey's validated age range.

The displacement CSV has 78 rows but only 52 unique event identifiers, including 74 recommended figures and four triangulation rows. Figures range from 3 to 116,969, with a median of 55.5, over dates from 13 January 2025 to 15 January 2026. These are event-scale figures, not 78 camp history. The population generator uses them as scale inputs for artificial histories. The original export/version chain remains unresolved under M07; no precise upstream attribution beyond the available local material is invented.

Earlier formula-label and quantile experiments have incomplete inputs and are excluded from principal findings. The 60-row controlled-profile log lacks complete message outcomes. No recruited participant study was supplied.

3.3 Experimental or study procedure

3.3.1 Network rescoring and operating points

The forests saved were rescored without retraining. The audit regenerated the trace, applied repository features and saved scalers, and reconstructed the chronological split. Argmax, persistence, and the saved thresholds were compared on that partition.

Both classifiers use 120 trees, maximum depth 10, minimum leaf size 20, balanced class weighting, and random seed 42. The state classifier has 25 inputs; the QoS classifier adds normalized message priority for 26. The state training code evaluates 15 threshold pairs and selects the largest escalation recall within a 15% false-alarm budget. Selection uses the evaluation partition, so threshold-policy results are exploratory operating-point evidence. Future labels extending across the split boundary were not purged in the source procedure. These two limitations are retained rather than silently corrected while quoting the old model's score.

QoS evaluation compares the saved actions with the generator reference using their shared probability function. Delivery excludes buffered actions; label accuracy, buffering and mean QoS retain separate interpretations. The reference differs from the runtime fallback.

3.3.2 Food-model evaluation

The household regression was independently refitted on the available derivative using six age-band counts and no intercept. Five-fold cross-validation uses shuffled folds with seed 0, matching the training script. Predictions from the held-out folds provide mean absolute error and coefficient of determination. A final fit on all derivative rows provides the reproduced coefficients. The source also fits a weighted model and creates bootstrap intervals, but the deployed service uses the unweighted coefficients. The present audit does not recreate the absent upstream preparation or independently rerun the saved coefficient intervals.

The child-model summary is reported without a new fit. Its code uses RidgeCV after standardizing inputs, but the scaler is fitted before the outer cross-validation loop. Full data mean and median constants also supply the baselines. Preprocessing should be fitted within each training fold to prevent held-out information entering the model path

[18]. Original child inputs are needed to measure the effect of correcting this procedure. The saved resampled-cohort MAPE is described together with its pool and aggregation size, rather than presented as a real-camp trial.

3.3.3 Population rolling-origin evaluation

The population back-test was reproduced with the script's default seed 42, 200 generated histories of 21 days per scenario and 6% noise. One scenario uses normal intake, and the other includes a generated mass-arrival burst. Each origin produces forecasts for horizons one through seven using only its available history. Comparators are flat hold, ordinary least squares over available history, undamped recent-window Theil–Sen and the damped version. This matches the general rolling-origin principle [11] while preserving the source's specific generated scenarios.

The seven-day horizon has 2,400 origin/horizon pairs per scenario; the one-day horizon has 3,600. These counts represent overlapping forecasts within 200 histories, not thousands of independently observed camps. The OLS comparison uses a different history window from the robust methods, so it evaluates the implemented configurations rather than isolating robustness alone. No tuning experiment or statistical significance test is reconstructed from an undocumented procedure. Full horizon results appear in Appendix A.

3.3.4 Software verification procedure

Selected cloud resource checks were executed directly to obtain assertion counts rather than relying on a runner that reported only passing files. The network Python suite exercises model and feature logic. Selected local network, priority, and sync-integration suites were executed against isolated temporary SQLite databases. An initial run used uninitialized fixtures; after schema initialization, 12 fixture-related failures disappeared. The reportable run has 77 passes and 11 failures. Both logs are retained, but the initial fixture errors are not counted as current policy failures.

3.4 Measures and evaluation metrics

Table 3.2 defines the principal measures and their units. Mean absolute error retains the unit of the target, making people, kilograms per household and kilograms per child visibly different quantities. Mean absolute percentage error is meaningful only when

the actual target is nonzero, and its averaging unit must be stated. Coefficient of determination compares squared residual variation with the target total variation; a low value indicates limited explained variation even when an aggregate prediction is useful.

$$\mathrm{MAE} = (1/n)\,\Sigma_i\,|y_i - \hat{y}_i|;\ \mathrm{MAPE} = (100/n)\,\Sigma_i\,|(y_i - \hat{y}_i)/y_i| \quad (3.1)$$

$$R^2 = 1 - \Sigma_i(y_i - \hat{y}_i)^2 / \Sigma_i(y_i - \bar{y})^2 \quad (3.2)$$

Network accuracy is the proportion of correctly labeled future states. Escalation recall uses only cases where the actual future state is more severe than the current state. Its numerator counts a predicted state more severe than the current state, so it measures warning of deterioration rather than exact destination-state identification. False-alarm rate uses non-escalating cases as its denominator. Persistence predicts the current state and therefore has zero escalation recall by construction.

$$\mathrm{Recall}_e = \text{caught escalations / actual escalations};$$
$$\mathrm{FAR}_e = \text{false escalations / non-escalating cases} \quad (3.3)$$

Table 3.2 Evaluation measures and denominators

Measure	**Unit / denominator**	**Permissible interpretation**
Accuracy	Correct labels / all evaluated rows	Generated label agreement
Escalation recall	Warning / 467 actual escalations	Detects some worsening; not exact future-class recall
Escalation false-alarm rate	False warning / non-escalating rows	Unnecessary warning burden
Conditional delivery	Mean assumed probability / sent actions	Excludes buffered work
Buffering proportion	Deferred actions / all actions	Immediate withholding, not eventual loss
MAE / MAPE / bias	Target unit / % / target unit	Horizon- and dataset-specific error
R^2	Residual variation relative to target variation	Explained variation under the model
Test passes/failures	Executed assertions	Selected fixture behaviour

For a policy action, the conditional delivery score averages the generator's probability only over actions that transmit. Buffering is reported over all evaluated actions. This makes the selected denominator visible: an apparent increase in conditional delivery may occur alongside a decrease in the fraction sent immediately. The mean QoS is a

transport-action summary, not a bandwidth or energy measurement. Those outcomes need independent traffic and hardware observations.

3.5 Data analysis

Comparisons retain their setting and units. Conditional delivery differences are percentage points; relative MAE reduction divides the error difference by the comparator error. Overlapping forecast origins are not treated as independent camp observations.

3.6 Ethics of privacy security and responsible Registration, identity and demographic information may be sensitive in a relief setting. The audit used source and derived research artefacts rather than exporting runtime personal records. Deployment would require an agreed access model, retention rules, consent, or another documented basis for processing, and restricted research access. The presence of TLS configuration or password fields does not establish that those controls are correctly deployed. No ethics approval number, security certification or consent procedure is invented.

The planning outputs should remain subject to human review. A low-stock warning is an estimate based on recorded balances and assumed demand; it cannot determine nutritional adequacy or guarantee supply. Message priorities also express value judgments about whose information is delayed. Responsible use requires the policy to be explainable and reviewable, including the consequences of expiry and unavailable inputs. Current generative AI assistance in source analysis, drafting and document production is disclosed in Appendix B; historical assistance requires author confirmation.

3.7 Reproducibility and research artefacts

The evidence package retains source/template hashes, working-tree context, metrics, and logs. Appendix A gives extended results and contracts; the evidence map identifies reproduction paths.

4 PRODUCT AND SOLUTION DESIGN

4.1 Requirements

The design requirements are reconstructed from the implemented interfaces and evaluation objectives. They are not presented as requirements elicited from a documented participant study. The resource path must accept current population and history, make cohort and rate assumptions visible, calculate food and drinking-water demand, and relate demand to compatible inventory quantities. The network path must distinguish message importance, receive recent observations, select a supported publishing action, and retain deferred work according to a defined queue policy. Table 4.1 gives acceptance evidence for these requirements and identifies incomplete validation.

Table 4.1 Reconstructed requirements and available verification

Requirement	**Implemented mechanism**	**Acceptance evidence**
R1: Explain demand basis	Cohort rates and forecast metadata	Code inspected; no field demand trial
R2: Forecast population	Damped recent-window robust trend	Generated horizons 1–7 evaluated
R3: Compatible stock use	Units, commodity matching and ledger	Pure tests; live database trial missing
R4: Prioritise publishing	Priority + state/model QoS matrix	Code and selected tests; expectations disagree
R5: Retain pending work	SQLite queues, retry and expiry	Limited fixtures; outage trace missing
R6: Track cloud acceptance	sync_id and application ACK	Contract inspected; full replay trial missing
R7: Expose current state	HTTP status and SSE	Interface present; complete UI build unresolved

Several nonfunctional requirements follow the operating context but need measured acceptance criteria before deployment. A local application should tolerate a temporary cloud disconnection; forecast outputs should expose their input freshness, and stock adjustments should preserve a consistent balance. The code contains mechanisms relevant to these requirements, including SQLite queues and PostgreSQL transactions. Their existence supports a design account; acceptable outage duration, recovery latency and concurrent-user capacity have not been established experimentally.

4.2 Component architecture within the complete solution

Figure 4.1 shows the source-derived logical architecture. ESP32 sensor and fingerprint code communicates with a Python Fast API edge gateway and local operational services. The camp platform uses an Express backend and React/Vite interface, with SQLite stores for local state. A separate Python process measures network conditions and posts decisions to the backend. Adaptive MQTT and synchronization carry selected records through an EMQX broker to the cloud Express service. Cloud planning uses roster and snapshot information together with PostgreSQL inventory records; a Next.js interface exposes those outputs.

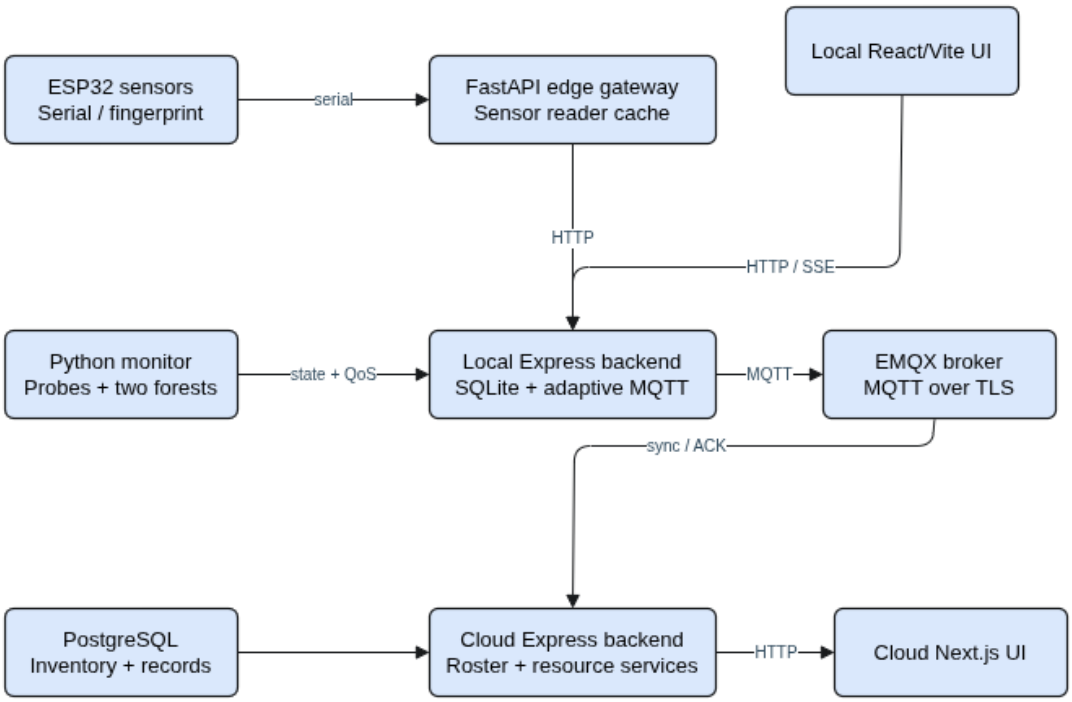


Figure 4.1 Source-derived system architecture and principal protocols

This separation assigns different responsibilities to measurement, decisions, and durable records. The Python monitor can load scikit-learn artefacts without embedding Python inference in the Express process. The backend remains responsible for queuing and publishing decisions that require local application to state. Cloud forecasting is implemented directly in JavaScript rather than depending on the adjacent education Flask service. The architecture is heterogeneous, but its interfaces make the language boundaries explicit.

Figure 4.2 expands the two component paths. Network state and QoS decisions enter the adaptive transport module, while row classification and sync identifiers enter the synchronization manager. Resource services obtain live camp information, forecast population, estimate demand and calculate stock outlooks. The component diagram includes the older local resource-request adapter as a separate path because its expected external responder was not found. Joining it visually to the cloud forecaster as a verified response contract would misrepresent the source of evidence.

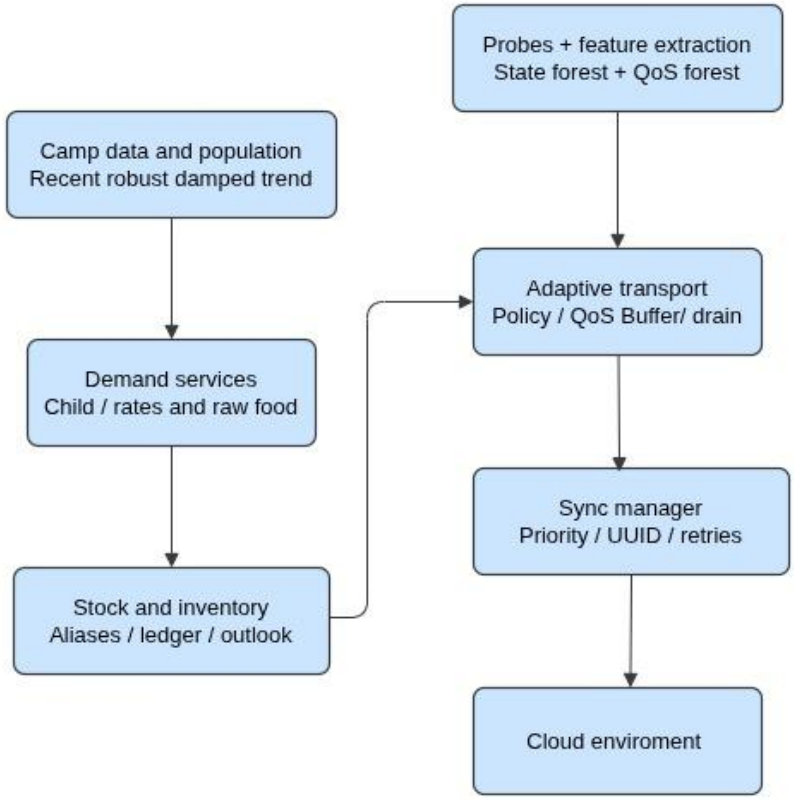


Figure 4.2 Component dependencies and the separate legacy prediction adapter

4.3 Design alternatives and decisions

The population service chooses a short recent trend instead of fitting an unconstrained long-history line. A recent window can respond to changing intake, and the median pairwise slope reduces the influence of an isolated extreme observation. Damping limits continuing extrapolation, while a population cap prevents unbounded output. These are reasonable design motives inferred from the mechanism. The back-test compares resulting configurations, but it does not isolate each choice or prove that the same parameters are optimal for a real camp.

The network design combines a probabilistic predictor with an observation-based safety floor. A future-state model may detect deterioration before a current threshold is crossed; current evidence can nevertheless require a more severe immediate state. The effective state therefore takes the greater severity of the two decisions. Recovery hysteresis then prevents one improved sample from causing immediate repeated flushing. This combination prioritises conservative transitions while accepting additional buffering. The appropriate trade-off depends on real message deadlines, which are not specified by the simulation.

Transport policy has two alternatives in the source: a fresh model policy or a deterministic fallback. The model can assign QoS 0, 1, 2 or buffer for each priority. The fallback maps state and priority to a rule. A 90-second freshness limit avoids indefinite reuse of a stale decision. If no recent state remains, transport connectivity provides the fallback state. This decision is simple and inspectable, but a connected socket is not evidence that the path is suitable for all application workloads.

Inventory storage uses transactional balance updates and a movement ledger rather than deriving present stock only from a changing forecast. Recorded stock and predicted use therefore have different roles: the former is an operational balance, the latter a planning estimate. This permits an operator to adjust a quantity and retain its reason while recalculating the outlook. Database locking is a design mechanism for competing updates, consistent with PostgreSQL row-lock semantics [19]; its concurrent behaviour still needs a database-backed experiment.

4.4 Detailed design

4.4.1 Network measurement and feature construction

The monitor obtains ping round-trip time, loss and jitter, three broker TCP connection timings, bridge reachability and local queue depth. A bounded probing cycle limits broker connection attempts. Queue delta describes change in pending work. These inputs describe timing, reachability and backlog; they do not directly measure throughput or available link bandwidth. Figure 4.3 presents the implemented path from probes to the HTTP state update.

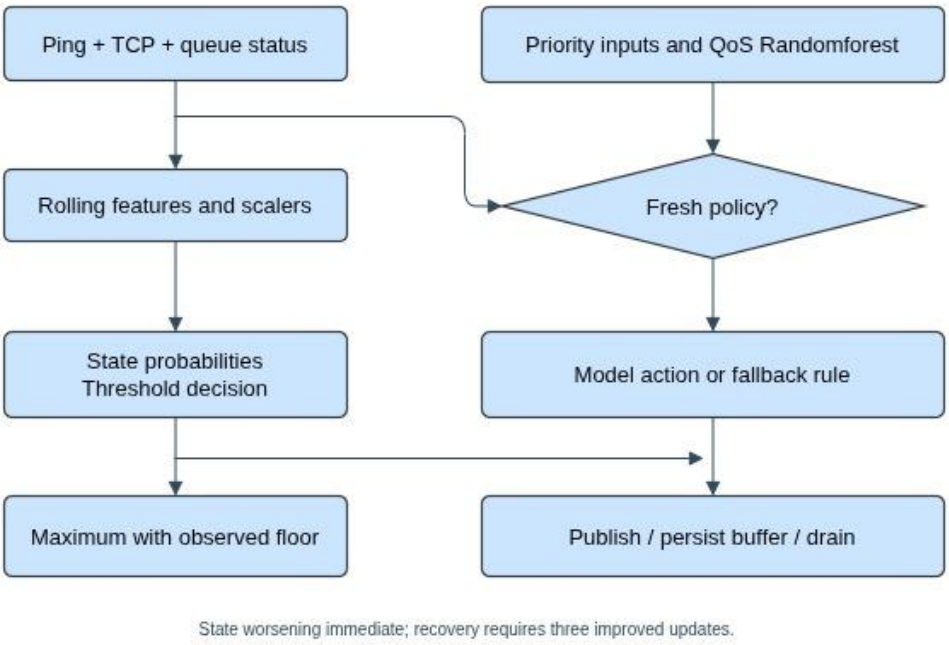


Figure 4.3 Network measurement, prediction and effective-state workflow

The 25-feature vector begins with eight direct inputs. Five selected metrics also contribute rolling means and endpoint slopes over a five-row window. Additional features include round-trip-time standard deviation, bridge uptime fraction, the interaction between loss and latency, and sine/cosine encodings for hour and day of week. For a windowed metric x, the slope is $(x_t - x_{t-4})/4$. It is an endpoint change per sample, not a least-squares trend. Priority divided by three extends this vector for QoS classification.

$$\text{mean}_5(x_t) = (1/5)\,\Sigma_{j=0}^{4}\, x_{t-j};\ \text{slope}_5(x_t) = (x_t - x_{t-4})/4 \quad (4.1)$$

The classifier predicts probabilities for stable, degraded and critical future states. The operating rule first checks whether critical probability reaches its threshold, then whether degraded plus critical probability reaches the degraded threshold. The selected saved values are both 0.25. The live observation floor uses a partially different threshold set, including critical queue depth of 100 and degraded queue depth of 50. Consequently, classifier accuracy alone cannot describe the complete live decision. The monitor posts both diagnostics and a four-priority policy for application use.

4.4.2 Priority policy and queue behavior

Table 4.2 gives the current fallback matrix. Under stable conditions, the base levels are LOW 0, MEDIUM 1, HIGH 1 and CRITICAL 2. Degraded conditions upgrade high and critical work to QoS 2, while low and medium use QoS 1. Under critical conditions only critical work is sent by the fallback, at QoS 1, with the remaining priorities buffered. The learned policy may differ when it is fresh. The generator reference used in the experiment must therefore be named separately.

Table 4.2 Current deterministic MQTT fallback policy

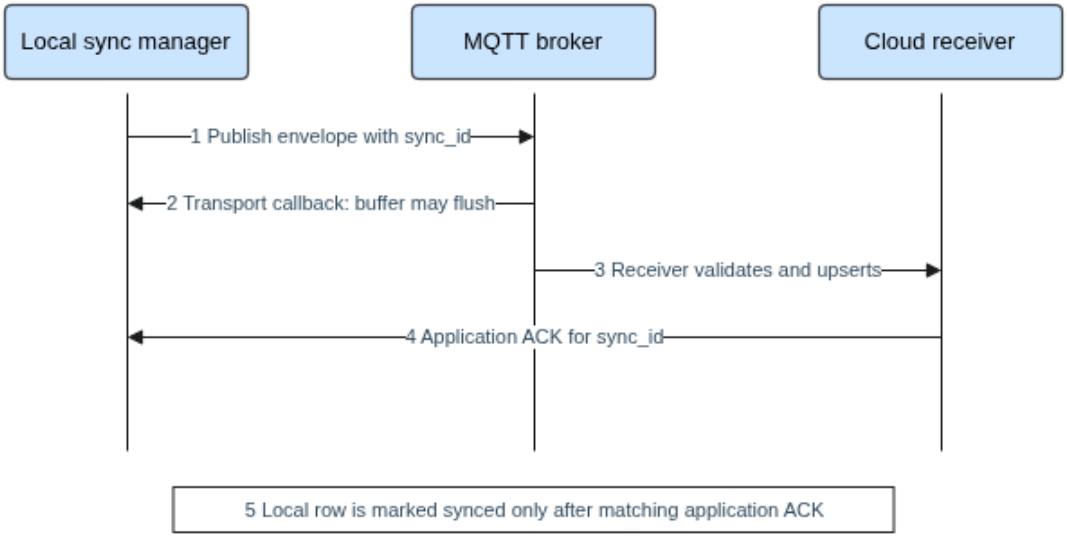
Message priority	**STABLE / DEGRADED QoS**	**CRITICAL state**
LOW	0 / 1	Buffer
MEDIUM	1 / 1	Buffer
HIGH	1 / 2	Buffer
CRITICAL	2 / 2	1
Fresh model policy	May choose 0, 1, 2 or buffer	Overrides rules while valid

Operational classification and row-content classification are separate functions. Registration is currently critical, check-in high and routine distribution medium in the operation path. Person synchronization rows are critical in the row classifier, and the sync manager passes an override to publishing. Some current tests expect earlier values. This is a requirements consistency problem because a correct queue order still depends on correct assignment of importance. Chapter 5 reports the disagreement rather than altering the design description to match the tests.

The persistent message buffer defaults to 5,000 rows. Admission can evict the oldest row among the least important eligible work or reject a new row when its importance does not justify eviction. Low and medium records expire after 24 hours; flushed rows are retained for 60 minutes. Drainage orders critical, high, medium and low, then oldest first within a priority. Defaults are 50 rows per batch with a 250-ms pause. These finite bounds explain why persistence cannot be described as a universal no-loss guarantee.

Worsening state changes are applied immediately. Recovery needs three successive improved updates, and periodic sweeps and reconnect handling can trigger drainage. A publish callback or timeout determines whether the buffered row is marked to flush.

Failure leaves work pending or buffering it. The application still needs a separate cloud acknowledgment for synchronization completion. Figure 4.4 shows those distinct events and the point at



which the local synced flag changes.

Figure 4.4 Synchronization sequence with transport completion and cloud application ACK

4.4.3 Population and demand calculation

The population forecast uses the last seven observations when available. For distinct observation times t_i and t_j, the slope is the median of pairwise changes, and the intercept is the median of residual offsets. The forecast is anchored to the current population and adds damped increments with $\varphi = 0.8$. Projection is constrained to the interval from zero to three times the current population. Fewer than three observations cause a flat hold; three to six produce a medium history-length label; seven or more permit the source's high label and residual envelope.

$$\hat{b} = \text{median}_{i<j}[(P_j - P_i)/(t_j - t_i)]; \hat{a} = \text{median}_i(P_i - \hat{b}t_i) \quad (4.2)$$

$$\hat{P}_{t+h} = \text{clamp}[P_t + \hat{b}\,\Sigma_{k=1}^{h}\varphi^k, 0, 3P_t], \text{ with } \varphi = 0.8 \quad (4.3)$$

The envelope grows from median absolute residual spread as the horizon increases. It is a heuristic scenario range, with no verified nominal coverage. The labels LOW, MEDIUM, and HIGH describe history availability rather than the probability that an outcome lies in the range. Time gaps also matter: seven observations need not be seven

consecutive calendar days. The service's day-zero value represents the current anchor, after which projected values feed the demand calculation.

Figure 4.5 connects the resource stages. Served-food planning uses child predictions where individual child records are supplied, an explicit override when configured, or a default child rate. Adult and older-person quantities use configured or ratio-derived rates. A pregnancy top-up adds food to the applicable base quantity. The projected population is combined with a fixed composition unless input records change. Drinking water is a separate per-person calculation, keeping litres outside the food-mass arithmetic.

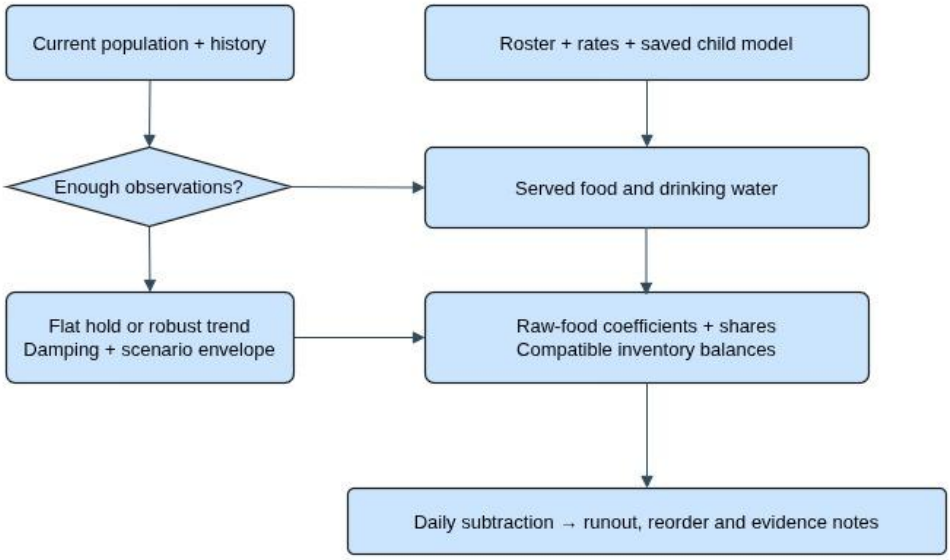


Figure 4.5 Population, cohort demand and commodity stock workflow

$$F_{served,h} = F_{c,h} + r_a N_{a,h} + r_e N_{e,h} + r_{75} N_{75,h} + r_p N_{p,h};\ W_h = r_w \hat{P}_h \quad (4.4)$$

Here Fc,h is the child total, and the remaining terms denote adult, older-person and pregnancy additions. Table 4.3 records the deployed quantity bases. The 1.6-kg adult rate comes from configured meal masses of 0.5, 0.6 and 0.5 kg. It is not an independently measured national intake. Older-person served rates apply survey-derived relationships to that configured base. The drinking-water constant is 3 L, and the cloud path applies no pregnancy water top-up on this basis.

Table 4.3 Planning rates, units and evidential basis

Quantity	Planning value	Basis
Child served food	1.76 kg/person/day fallback	Saved survey median, or individual model/override
Adult served food	1.60 kg/person/day	Configured meal masses
Older served food	1.71 (60–74); 1.37 (75+); blended 1.63 kg	Survey relationships applied to configured adult basis
Pregnancy food addition	0.60 kg/person/day	Configured top-up
Cloud drinking-water allowance	3 L/person/day	Narrow standards-derived planning assumption
Cloud pregnancy water addition	0 L on this basis	No transferred domestic-water top-up
Raw solid food	Six reproduced coefficients	Household-derived stock basis; Table 6.3
Local water issue configuration	15 L/person/day	Broader basis; not interchangeable with cloud allowance

4.4.4 Raw food, commodity mapping and stock depletion

The raw-food model calculates a household total as the sum of age-band counts multiplied by saved coefficients, with no intercept. Runtime roster ages select the appropriate coefficient. Unknown ages use a survey-weighted composition mean; if no roster exists, the service uses the survey composition. These fallbacks permit calculation while introducing explicit assumptions about the missing population. They do not establish that an unrecorded camp has the same composition as the survey.

$$F_{raw} = \Sigma_g \beta_g N_g, \text{ for six age bands } g \quad (4.5)$$

Commodity standardization strips packaging text applies curated aliases and group matching, and classifies food, water, beverages and nonfood items. The mechanism is deterministic matching rather than learned natural-language understanding. Commodity shares allocate estimated raw food to available categories, with claimed sets used to avoid duplicate group allocation. Unknown food receives a residual-share assumption. This assumption affects depletion estimates and requires actual issued quantities to validate it.

For each day, the stock outlook subtracts compatible commodity demand from the starting balance. The first strictly negative remaining balance is the runout event; reaching exactly zero does not trigger that condition. The service repeats the

calculation for midpoint and envelope paths, then subtracts a lead time, default three days, from the estimated runout date to obtain a reorder date. Warning classes compare coverage with one and two lead-time intervals. A date beyond the horizon is unresolved within that horizon, not an assertion that stock is indefinitely sufficient.

$$S_{k,h} = S_{k,0} - \Sigma_{d=1}^{h} D_{k,d};\ h_{runout} = \min\{h : S_{k,h} < 0\} \quad (4.6)$$

Inventory adjustment accepts food mass in kilograms and water in liters or milliliters, converting milliliters by 0.001. Receipt and issue signs are enforced, and original and base units are retained. The repository records a movement and balance update within a transaction and rejects a negative balance. Prediction snapshots preserve JSON input/output context. Figure 4.6 shows the logical data responsibilities without implying that local and cloud tables are a single database.

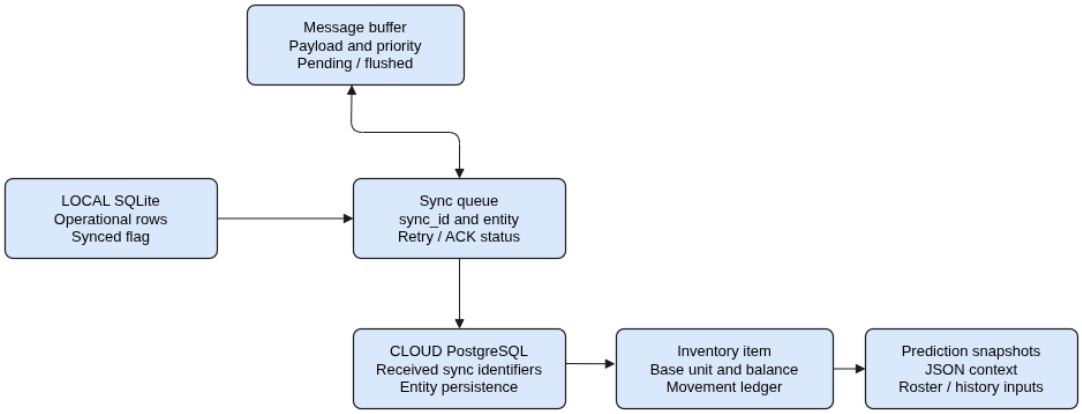


Figure 4.6 Logical, persistent records for queues, synchronization and inventory

4.5 Standards and professional practice

The design combines defined transport semantics with source-specific application contracts. Reviewable outputs need identifiers, input dates, quantity bases and assumptions; recorded inventory movements and matching sync acknowledgments provide auditable completion points. Current end-to-end gaps remain part of the assessment.

Environment secrets are excluded from research artefacts. Paths and hashes identify inspected evidence while preserving attribution. Deployment of security and institutional records require the separate validation identified in Chapter 3.

5 IMPLEMENTATION INTEGRATION AND TESTING

5.1 Development environment and technology stack

The implementation combines JavaScript, Python and Arduino C++ across independently defined services. The local backend uses Express, better-sqlite3 and MQTT.js. Its frontend declares React 18 with Vite and charting dependencies. The cloud backend uses Express and PostgreSQL configuration, while its frontend declares Next.js 16.2.1 and React 19.2.4. The Python network environment declares NumPy, pandas, joblib and scikit-learn. Table 5.1 separates manifest information from the execution environment used for the audit.

Table 5.1 Implemented stack and verified execution context

Layer	Technologies	Version/evidence context
Sensor / gateway	ESP32 Arduino C++; Python FastAPI	Source and manifests
Local application	Express; better-sqlite3; MQTT.js; React/Vite	React 18 declared
Network models	Python; pandas; NumPy; joblib; scikit-learn	scikit-learn 1.9.0 used in audit
Cloud application	Express; PostgreSQL; Next.js; React	Next 16.2.1 / React 19.2.4 declared
Reproduction runtime	Node.js and Python	Node 20.20.2; Python 3.12
Deployment definitions	Docker Compose; Nginx; host-network monitor	Configuration only; hardware measurements missing

The audit used Node.js 20.20.2 and scikit-learn 1.9.0. These versions describe the reproduction environment, not an asserted field of installation. Docker Compose definitions, service commands and a target-board reference do not establish a physical host's RAM, operating-system image or network interface. The thesis consequently records those deployment measurements as missing. No inference-time, energy or memory-use figure is inferred from the model's file size.

5.2 Component implementation

5.2.1 Training artefacts and runtime monitor

The state training path in network_predictor.py calls feature extraction, maps state names to severity codes and splits chronologically. StandardScaler is fitted on the training portion and saved with the forest and feature list. Scaling is not mathematically

required for decision-tree splits, but the saved pipeline must be respected at inference to maintain consistency with the fitted artefact. The QoS path follows a corresponding construction with priority included. The audit loads only the trusted local project artefacts; it does not treat an arbitrary model file as safe executable data.

The state model occupies 4,671,041 bytes and the QoS model 2,904,569 bytes, with 25 and 26 input dimensions. These are verified artefact properties, not measured hardware memory or inference performance.

network_monitor_service.py performs the probes, maintains recent history and invokes both models. Broker timing uses TCP connection attempts with a two-second connection timeout, and a six-second bounded deadline across three probes. That test detects endpoint reachability and connection timing, not authenticated with MQTT publication. Queue depth is read from the local backend, allowing application backlog to influence the network features. A missing or failed probe therefore needs careful interpretation: reachability, service availability and link quality are related but separate conditions.

The monitor forms a probability-threshold state, compares it with the observed safety state and posts the more severe result. Its payload includes measurements, diagnostic probabilities, decision information, and a four-priority QoS policy. The default interval is 30 seconds, also used in Compose, but the process sleeps after measurement and inference. Work duration therefore adds to the interval. A command-line override changes cadence again, so the generated ten-sample horizon cannot be described as an exact five-minute live forecast.

5.2.2 Adaptive transport and persistent synchronization

saamp-mqtt.js implements the state transitions, policy selection, buffer admission and drainage. Recent model policy takes precedence where valid; expired policy falls back to the state/priority matrix in Chapter 4. State recovery requires repeated improvement. A 30-second periodic sweep, reconnection events and recovery can trigger flushing. The ten-second publish timeout protects the drain operation from waiting indefinitely on a callback. Batch size and pause defaults bound one sweep's work, though they do not establish a measured fairness or recovery guarantee.

The message buffer and sync queue serve different purposes. The buffer retains a publication awaiting transport handling. The sync queue retains the identity and status of a logical record that requires cloud acceptance. An envelope carries a UUID *sync_id* and entity content on *smartcamp/sync/<entity>*. The cloud receiver uses that identifier for duplicate handling and sends an application to ACK after its processing path. The local manager associates the ACK with pending work before marking the originating record synced. A transport callback alone cannot make that state transition valid.

Retry logic begins at 60 seconds and increases exponentially up to 1,800 seconds, with ten retries configured. Acknowledgment checks run every 15 seconds. Synchronization periods vary between 5, 15 and 60 seconds according to the state. The manager pauses new publishing under critical conditions, while pending work retains its own retry lifecycle. This layering explains why the outcome of the whole system cannot be predicted solely from the QoS classifier's action. At fault, restart and replay trials remain necessary to test interacting timers and queues.

5.2.3 Population and child-intake services

populationForecast.service.js normalizes dates, fits its trailing observations and returns daily values together with method, history-length confidence, slope, observation count, trend window, damping and notes. Keeping those fields with the forecast helps downstream code explain a cold start hold or a growth cap. A zero current population requires particular care: the code only applies its upper ceiling when that ceiling is positive, so the general positive-population cap is not a universal invariant for every degenerate input. This source-specific detail is considered in future edge-case validation.

childIntakeModel.service.js loads the saved RidgeCV JSON and performs the same standardization using stored means and scales. Central is the reference province, with dummy inputs for Eastern, Northwestern, Southern and Western. The model's intercept is approximately 1.9136 kg and its saved regularization parameter is 5.623413. Runtime age values are clamped to 6–14. Missing or unsupported province information is handled using the service's fallback encoding rather than a newly trained province model. The prediction is a scalar dietary mass; it does not estimate an individual child's nutritional requirement.

$$\beta_{ridge} = \arg\min_{\beta} [\Sigma_i(y_i - \beta_0 - z_i^T\beta)^2 + \lambda\Sigma_j \beta_j^2] \quad (5.1)$$

Equation 5.1 defines the saved model's penalized linear objective. Runtime evaluates stored coefficients; distributionPlan.service.js selects child records, an override or a default, then combines cohort rates along the population forecast.

5.2.4 Raw food, stock and inventory services

rawFoodRates.service.js consumes the deployed unweighted age-band coefficients. It distinguishes adult and older-person bands and supplies composition fallbacks for incomplete rosters. commodityStandardizer.service.js uses curated aliases, packaging removal and category rules; commodityShares.service.js maps demand into stock groups. The inspected rule set contains 67 aliases, 98 nonfood entries, and 13 beverage entries, while the share artefact contains 76 entries. Selected tests check specific matching and classification cases. These counts describe the inspected rule set, not classification accuracy on independently sampled stock descriptions.

stockRunout.service.js calculates daily commodity demand against recorded starting stock. For population extrapolation, it can replicate a roster and take a prefix for a partial replication. That implementation makes the partial cohort dependent on roster order, which can bias composition when records are sorted by age or household. Unknown and nonconsumable items receive limited or unavailable outlooks rather than a learned medical/shelter estimate. The default stock horizon is 30 days, with other inventory-dataset paths allowing up to 90; neither duration is validated by the seven-day population experiment.

inventory.service.js normalizes supported units, enforces movement direction and delegates durable changes to inventory.repository.js. The repository wraps balance changes and movement insertion in a transaction, selects the relevant row for update and rejects a resulting negative balance. The ledger retains the adjustment context, and prediction of snapshots store structured JSON. A transactional source path is stronger than an unprotected read modify write design, but this audit pure tests do not establish rollback and locking behavior under simultaneous database clients.

inventoryPrediction.service.js constructs a prediction-input dataset from inventory items, roster/household groupings and population history. It also supports saving

snapshots. The name prediction dataset does not mean that the service trains online. No automatic retraining on accumulated camp movements was found in this path. A future feedback loop would need targets with an agreed consumption basis, a preparation procedure and a validation schedule before new coefficients could safely replace the saved ones.

5.3 Integration with the other components

Table 5.2 summarizes the main API groups. The network controller exposes state, status, server-sent-events stream, flushing, and test publishing. The local interface subscribes to updates through Event Source. The HTTP update joins the Python monitor to JavaScript policy handling, while MQTT joins local and cloud services. These are different integration channels with different failure meanings. An HTTP success from the local controller does not establish that the next message reached the cloud.

Table 5.2 Principal network and resource integration contracts

API group	Selected routes	Role
Local network	/api/network/state; /status; /stream; /flush; /test-publish	Monitor update, diagnostics, SSE and queue control
Cloud planning	/api/resources/overview; /distribution/today; /distribution/plan; /distribution/rates	Roster/attendance and daily demand
Cloud stock	/api/resources/stock/outlook; /stock/alerts; /stock/aliases	Depletion estimates and matching evidence
Cloud inventory	/api/resources/inventory/items; /items/:id/adjust; /summary; /categories	Ledger inputs and current balances
Prediction input	/api/resources/inventory/prediction-dataset; /prediction-snapshots	Construct and retain calculation context
Legacy Pi adapter	External /predict/batch; /predict; MQTT forecast_request	Responder not identified in deployable source

The cloud sync receiver persists supported entities and bridges persons and wristbands into the live camp store. campData.service.js supplies roster and snapshot information for planning, and campStore.repository.js supplies persistence. Some live maps are keyed by local numeric person identifiers despite camp identifiers appearing elsewhere in the schema and payload. The audit has not established cross-camp isolation for

colliding with local IDs. Multi-camp support is consequently an integration intention requiring a focused test, not a validated scalability result.

Tomorrow's overview combines population projection with attendance history. A weighted ratio from usable check-in/headcount information is bounded between 0.5 and 0.99; absent usable history defaults to 0.85. Child records may then be selected by a prefix for the attendance-adjusted estimate. This gives an explicit operational rule, but no independent attendance study validates the fallback or the order-based subset. The overview and the full distribution plan should therefore be interpreted with their respective population bases visible.

The Pi resource prediction adapter is an older, separate integration. It calculates current population and recent mean arrivals, extrapolates linearly, then attempts at an external batch HTTP prediction endpoint, an individual endpoint and an MQTT forecast request. The repository contains a listener's contract for a result, but the corresponding deployable resource responder was not found. The adjacent Flask education service is not that responder. This missing contract prevents a claim that every local resource screen is connected to the current cloud robust forecast.

5.4 Verification and software testing [Need to complete in final thesis]

The verification strategy combines numerical reproduction, selected unit and integration-style tests, and source review of boundaries that tests do not reach. Cloud checks cover inventory of pure functions, commodity standardization, stock forecasting, attendance defaults and prediction output structure. Python suite checks feature/model logic. The Pi suites cover state, message priority, and selected synchronization interactions. Table 5.3 reports assertion counts from the reviewed logs rather than a stale draft or file-level summary.

Table 5.3 Selected software checks executed during the audit

Suite	**Passed / failed**	**Fixture scope**
Cloud inventory	Fill in final thesis	Fill in final thesis
Cloud standardization	Fill in final thesis	Fill in final thesis

Suite	Passed / failed	Fixture scope
Cloud stock	Fill in final thesis	Fill in final thesis
Cloud prediction	Fill in final thesis	Fill in final thesis
Pi network	Fill in final thesis	Fill in final thesis
Pi priority	Fill in final thesis	Fill in final thesis
Pi sync integration	Fill in final thesis	Fill in final thesis
Pi selected total	Fill in final thesis	Fill in final thesis

Table 5.4 Current local policy-test disagreements and their interpretation

Policy area	Expected in failing assertions	Current source result
Operation check-in (1)	CRITICAL, QoS 2	HIGH, QoS 1
Operation distribution (1)	CRITICAL, QoS 2	MEDIUM, QoS 1
Person rows (3)	HIGH for selected ages/null	CRITICAL for all person rows
Distribution rows (4)	Medicine/paracetamol CRITICAL; water/food HIGH	MEDIUM
Sync integration (2)	Critical medicine; registration HIGH in order	Medicine MEDIUM; registration CRITICAL

Passing fixtures support their selected calculations. Live PostgreSQL concurrency, complete cloud replay/fault behavior and frontend build remain unverified. The existing resource interface conflicts therefore limit integration readiness despite passing backend checks.

5.5 Deployment and operation

Figure 5.1 presents the service arrangement derived from Compose definitions. The cloud includes backend/frontend services and an Nginx reverse proxy, while the local configuration separates the monitor and backend. Host networking in the monitor definition permits host-relevant probing, but it also ties the measurement context to the host environment. Environment variables configure service endpoints, database paths and broker access. Secrets and actual broker credentials are not reproduced in the thesis.

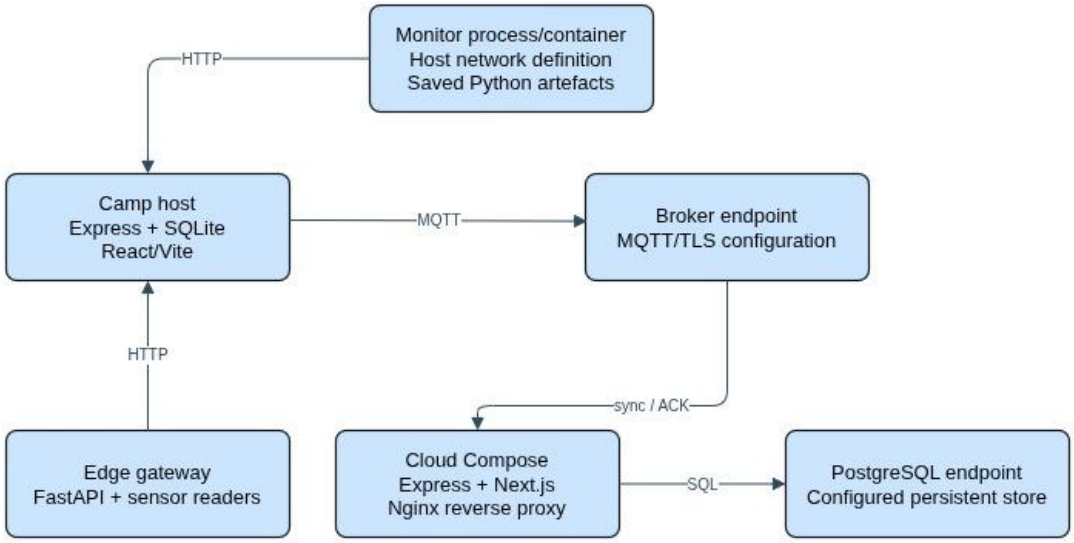


Figure 5.1 Deployment definitions and operational service boundaries

Operations should expose separate startups, monitor-readiness, and cloud-acceptance states. Queue depth, stale policies, retry exhaustion, and balance anomalies need monitoring. The available status interfaces provide a basis for this work, but no measured recovery or service-level objective was supplied.

6 RESULTS AND VALIDATION

6.1 Research results

6.1.1 Network-state prediction

The saved state model reproduced 92.40% argmax accuracy on the 7,998-observation generated evaluation partition. Persistence reproduced 93.57%, giving the model a difference of approximately −1.18 percentage points. The model therefore does not outperform persistence on overall accuracy. This outcome is compatible with the generator's strong temporal persistence: many future windows retain the present state. Table 6.1 reports on the operating-point comparison alongside the deterioration measures defined in Chapter 3.

Table 6.1 Reproduced network-state results on the generated evaluation partition

Decision	**Accuracy (%)**	**Escalation recall / FAR (%)**
Persistence	93.57	0.00 / 0.00
Forest argmax	92.40	32.76 / 3.23
Saved thresholds 0.25 / 0.25	84.65	49.04 / 12.28
Evaluation context	7,998 generated rows	467 escalations; thresholds selected here

There were 467 actual escalating examples. Argmax detected 32.76% by the source's warning definition, with a 3.23% false-alarm rate among non-escalating examples. The saved threshold policy increased escalation recalls to 49.04% and false alarms to 12.28%, while overall accuracy fell to 84.65%. Relative to argmax, this is an increase of 16.28 percentage points in warning recall at a cost of 9.05 percentage points in false alarms. It demonstrates a policy trade-off under the generated trace rather than unqualified predictive improvement.

Figure 6.1 shows the 15 saved threshold combinations. Lower critical thresholds can raise warning recall sharply, but the associated false-alarm rates exceed the configured budget. The selected point is within the 15% budget on this partition. Because the same partition was used to select it, its performance cannot be treated independent of final-test evidence. Nor does the plot evaluate the live observation floor, state freshness or recovery hysteresis; those mechanisms can change the effective application state.

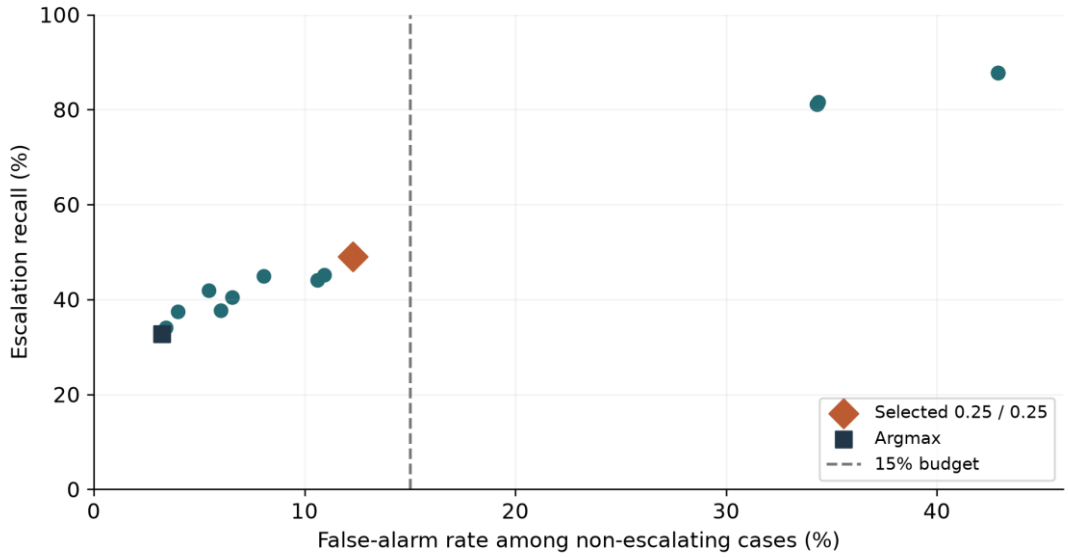


Figure 6.1 Saved threshold trade-off: escalation recall versus false alarms

The saved argmax class report has 4,645 stable, 2,697 degraded, and 656 critical examples. Critical precision is approximately 61.70% and recalls 64.33%, weaker than the other state classes. These class measures describe exact critical-label prediction, while escalation recall asks a different question about worsening relative to the present. Appendix A includes the class report so that the two interpretations are not conflated. No measured real-network forecast score is available under M09.

6.1.2 QoS classification and conditional delivery

The QoS model reproduced a 99.52% agreement with constructed action labels. This is classification accuracy, not 99.52% message delivery. The model's overall conditional simulated delivery was 90.08%, compared with 82.50% for the generator reference. Buffering increased from 0% to 12.23%, and average selected QoS increased from 1.140 to 1.375. Table 6.2 and Figure 6.2 show the priority-level results, including the proportion withheld from immediate transmission.

Table 6.2 Reproduced priority-level QoS simulation results

Priority	**Learned: delivery / buffer (%)**	**Reference: delivery / buffer (%)**
LOW	71.90 / 4.70	55.95 / 0.00
MEDIUM	93.13 / 4.71	82.98 / 0.00
HIGH	94.97 / 41.35	84.30 / 0.00
CRITICAL	93.20 / 0.00	93.20 / 0.00

Priority	Learned: delivery / buffer (%)	Reference: delivery / buffer (%)
Overall	90.08 / 12.23	82.50 / 0.00
Mean QoS among sent	1.375	1.140

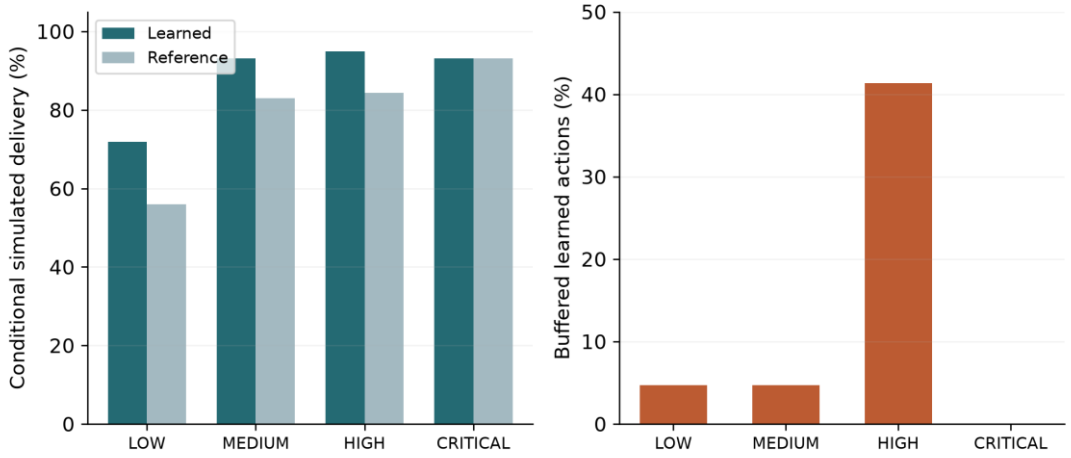


Figure 6.2 Conditional simulated delivery and buffering by message priority

The aggregate conditional delivery increase is 7.58 percentage points. Its denominator excludes buffered actions, so it cannot be interpreted as a gain in completed delivery over all messages. HIGH traffic is buffered in 41.35% of its evaluated actions; among transmitted HIGH actions, the probability average is 94.97%, just below the configured 95% target. Reporting only the aggregate delivery figure would conceal both this delay decision and the target shortfall.

CRITICAL traffic remains at 93.20% conditional delivery under both policies, below its configured 99% requirement. Its mean QoS is 2 and it is not buffered in the experiment. The prototype therefore shows no critical-delivery gain against this reference. The score also contains no observed bandwidth or energy cost, and the increase in mean QoS gives no basis for claiming such a saving. The shared probability assumptions and different runtime fallback restrict the comparison to its stated simulator setting.

6.1.3 Household-derived and child food estimates

The reproduced household model achieved five-fold mean absolute error of 1.1950 kg per household per day and R^2 of 0.0894 on the available derivative. It explains a limited proportion of household variation. Table 6.3 lists the fitted no-intercept coefficients,

including the adult rate of 0.8424 kg per person per day. Figure 6.3 plots these point estimates; it presents neither nutritional recommendations nor independently reproduced confidence intervals.

Table 6.3 Reproduced raw-food regression coefficients and validation

Age band / measure	Reproduced value	Unit / context
0–5	0.4669	kg/person/day raw solid food
6–14	0.4664	kg/person/day raw solid food
15–17	0.5702	kg/person/day raw solid food
18–59	0.8424	kg/person/day raw solid food
60–74	0.9148	kg/person/day raw solid food
75+	0.7593	kg/person/day raw solid food
Five-fold MAE	1.1950	kg/household/day
Five-fold R^2	0.0894	6,802 derivative households

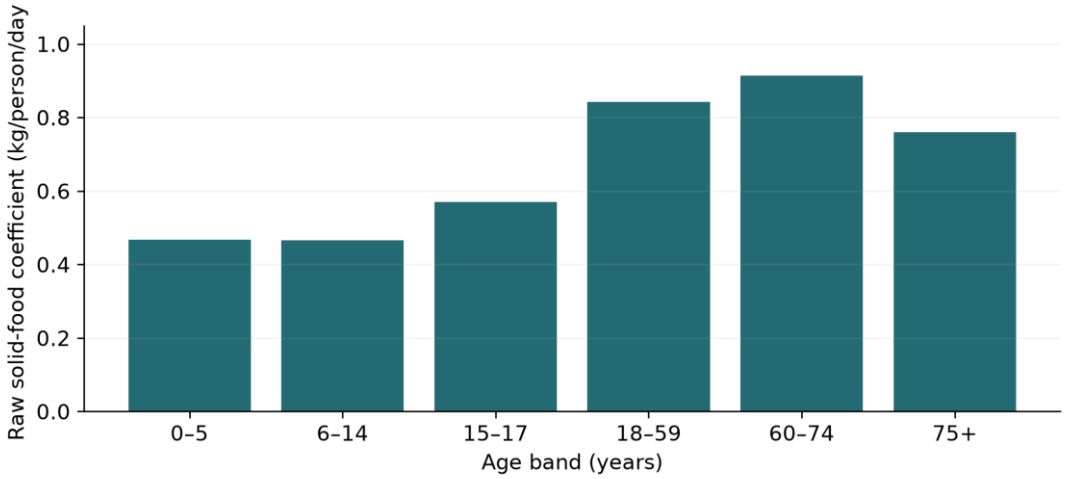


Figure 6.3 Reproduced age-band coefficients for raw-food planning

The coefficients describe the derivative and model specifications. Age counts alone explain little household variation; successful refitting does not complete upstream preparation or establish accuracy in communal camp cooking.

The child artefact reports five-fold MAE of 0.6191 kg per child per day and R^2 of 0.0679. These saved values were not independently rerun. Table 6.4 compares saved resampled-cohort errors. At a cohort size of 500, model MAPE is 1.500%, the mean-constant baseline is 1.514%, and the median-constant baseline is 8.090%. The improvement over the mean baseline is only 0.014 percentage points. A larger apparent

gain against the median should therefore not be attributed entirely to the covariate model.

Table 6.4 Saved child resampled-cohort MAPE and baseline comparison

Resampled cohort size	Model / mean-constant MAPE (%)	Median-constant MAPE (%)
25	6.631 / 6.823	9.235
50	4.480 / 4.654	8.408
95	3.363 / 3.471	8.004
200	2.282 / 2.363	7.993
500	1.500 / 1.514	8.090
Evidence	Saved artefact only	Resampled survey pool; not independent camps

Aggregation reduces some individual variation when the cohort is resampled from the same survey pool. That result does not establish performance in an independently observed camp or an age range outside 6–14. Full-data scaling and baseline construction also limit the saved comparison. A corrected score requires original child inputs; no corrected numerical result is invented. Drinking-water calculation has no learned-model error score because it applies an explicit allowance rather than a fitted water-consumption model.

6.1.4 Generated population forecasts

At the seven-day horizon in the normal-intake scenario, damped recent-window Theil–Sen reproduced MAE of 1,835.2 people, compared with 2,748.3 for flat hold and 3,096.1 for OLS. The corresponding reductions are 33.2% and 40.7%. In the generated burst scenario, its MAE is 1,763.4, compared with 2,744.8 and 4,405.5, giving reductions of 35.8% and 60.0%. Table 6.5 includes MAPE and bias, while Figure 6.4 displays MAE across horizons.

Table 6.5 Seven-day generated population forecast results

Scenario / method	MAE (people) / MAPE (%)	Bias (people)
Normal: damped robust	1,835.2 / 27.9	−1,104.5
Normal: flat	2,748.3 / 38.7	−2,748.3

Scenario / method	MAE (people) / MAPE (%)	Bias (people)
Normal: OLS	3,096.1 / 53.8	+949.5
Normal: undamped robust	2,698.9 / 45.0	+677.7
Burst: damped robust	1,763.4 / 30.0	−497.0
Burst: flat	2,744.8 / 40.9	−2,744.8
Burst: OLS	4,405.5 / 51.5	+2,904.6
Burst: undamped robust	3,422.2 / 44.2	+1,908.0

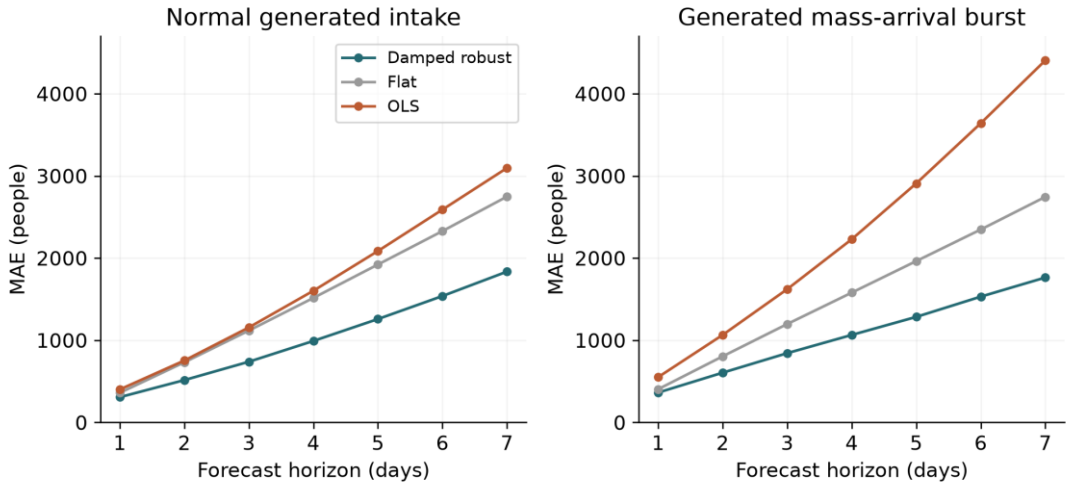


Figure 6.4 Population MAE by horizon for two generated scenarios

The damped method's errors still grow with horizon. Normal-scenario MAPE rises from 8.5% at one day to 27.9% at seven days, and seven-day bias is −1,104.5 people. Damping reduces over projection in this experiment but can also contribute to underprediction during sustained intake. The burst scenario's seven-day MAPE is 30.0%. A lower absolute MAE there does not imply that bursts are universally easier: scale and scenario construction affect absolute and percentage errors differently.

The undamped robust comparator has seven-day MAE of 2,698.9 in the normal scenario and 3,422.2 in the burst scenario. This supports the relevance of damping within the generated configuration. It does not isolate all causes of the difference from OLS because fitting windows also differ. The histories are artificial and scaled from event figures, with overlapping origins. No calibrated interval coverage or real-camp error reduction follows from these values. Appendix A reports the full horizon table for the principal methods.

6.2 Product and technical results

The source audit establishes implemented resource calculations, unit normalisation, commodity matching, persistent buffers and application acknowledgment handling. The software results provide selected executable evidence: 36 cloud assertions and 12 Python assertions passed; 77 of 88 selected Pi assertions passed. Eleven current policy disagreements remain. These outcomes answer implementation questions more directly than a broad claim that the entire product passed testing. The tests and their fixture limits were described in Chapter 5.

No measured inference latency, peak memory, CPU utilisation or power consumption is available for the physical target. Saved file sizes are reported as artefact properties only. The unresolved cloud frontend conflicts prevent a complete product build from being asserted. The thesis therefore presents an evaluated backend and model prototype with identified integration gaps, rather than a certified deployed product.

6.3 User or stakeholder validation [Need to add in final]

6.4 Integration results

Integration evidence is strongest at explicit contracts: the monitor-to-backend HTTP payload, local queue state, sync identifiers, the cloud application ACK path and cloud resource service composition. The local selected integration suite has 24 passes and two failures after fixture initialization. These fixtures do not exercise a real broker-to-cloud transaction under outages. The legacy prediction responder remains unidentified, database concurrency remains untested, and the frontend resource conflict remains unresolved.

End-to-end reliability would require a trace associating local creation, enqueueing, MQTT publication, cloud commit and local acknowledgment for the same sync_id. It would also need restart, duplicate and timeout cases. No such complete measured trace is supplied. Multi-camp collisions require a separate check because identifiers and live map keys have different scopes. These are concrete remaining integration tests rather than an unspecified request for additional testing.

6.5 Summary of findings

Table 6.6 answers the research questions within their evaluated settings. It distinguishes model comparisons, traceable calculation paths, and selected software behavior from the missing operational evidence.

Table 6.6 Research-question answers and current evidence boundary

Question	**Supported finding**	**Limit**
RQ1	Warnings improve at selected thresholds; accuracy trails persistence.	Generated trace; reused evaluation partition
RQ2	Conditional delivery rises; buffering and QoS increase.	No critical gain or complete delivery/byte score
RQ3	Damped configuration lowers generated h=7 MAE.	Artificial histories; unmatched OLS window
RQ4	Demand and stock paths have inspectable inputs and units.	Low dietary R^2; no independent camp trial
RQ5	Selected tests support some behaviors and reveal disagreements.	11 Pi failures; incomplete live integration

7 DISCUSSION

7.1 Interpretation of findings

The central result is that the prototype's value cannot be reduced to one accuracy percentage. Its resource services expose several assumptions that would otherwise be hidden inside a daily total, and its network services make importance and deferral explicit. The evaluation also shows why those mechanisms need outcome-specific measures. Network accuracy, warning recall, conditional delivery, household error and test passes describe different properties. Combining them into a single claim of success would remove the distinctions that make the study useful.

The state predictor illustrates the need for a strong simple baseline. Persistence performs well because most generated future windows resemble current conditions. A model can add warnings while making more mistakes on the majority of unchanged cases. The threshold rule accepts that trade-off, but its practical usefulness depends on the cost of missed deterioration and unnecessary buffering. Those costs were not measured. A critical warning that delays time-sensitive work could be harmful even if warning recall improves, so recall alone cannot select the deployed operating point.

The QoS result requires a similar denominator check. Conditional delivery rewards a policy for choosing to send under more favourable conditions, while deferral removes actions from the immediate delivery average. The model buffers a substantial portion of HIGH traffic and does not improve CRITICAL delivery in the simulated comparison. Its 99.52% label score mostly establishes that it learns the constructed mapping. A complete assessment needs eventual, deadline-qualified acceptance over all originated messages and the associated waiting, expiry and retransmission cost.

The resource results favour interpretability over strong individual prediction claims. Household and child R^2 values are low, although additive totals may still be useful as a first planning estimate. The saved child model's very small advantage over a mean constant suggests that a complex intake model should be retained only if independent operational evaluation justifies its inputs. The household coefficients give an understandable quantity basis, but the fit cannot resolve raw-to-served conversion, actual camp food preference or inventory recording error.

7.2 Comparison with literature baselines and existing solutions

The messaging literature motivates adaptation, but control location and validation setting remain decisive. Palmese et al. investigate MQTT-SN adaptation in simulation, whereas Jeong et al. describe broker-side requirements-aware control with equation-based evaluation [5] [6]. The camp prototype adds publisher-side forecasting, local buffers and application synchronisation. That combination is a product design difference, not evidence that its numerical results surpass either study. No common trace, protocol configuration or outcome measure permits a direct performance ranking.

The forecast comparison is consistent with the motivation for damped trends: indefinite continuation of a fitted slope can over project population. The generated results favor the damped recent-window configuration, but the strongest improvement against OLS combines several changes. A matched-window OLS model, an undamped OLS model, a damped OLS model and robust counterparts would help separate recency, estimator and damping effects. The existing undamped robust result gives one useful comparison, while a full ablation remains future work.

The framework also contrasts with a fixed-rate calculator. The prototype adds population projection, roster-aware rates and commodity mapping, allowing a stock outlook to react to changed inputs. That added behavior can improve explanation even before field superiority is established. However, the extra assumptions increase opportunities for error. A simple mean or configured rate is an appropriate baseline for a prospective camp trial, especially where model covariates are incomplete or their collection burden exceeds the demonstrated gain.

7.3 Research contribution

The research contribution supported by this draft is an evidence-based examination of an integrated prototype, with reproduced results and explicit limits. It connects source-level mechanisms to measures that retain their correct target and denominator. The network audit separates future-state classification from the complete runtime policy; the resource audit separates survey-derived mass from configured served rates and standards-based water. These distinctions support reproducible reasoning without claiming a new statistical algorithm.

Reproduction verifies available artefacts and identifies methodological weaknesses, including threshold reuse and child preprocessing before outer CV. Their corrected effect remains unmeasured. The contribution is documented evidence, not an invented improved score.

7.4 Product contribution

The product contribution lies in joining planning functions to operational records and communication state. The cloud services expose forecast metadata, rate bases, commodity matches and stock warnings. The local platform retains deferred work and tracks application acceptance through sync identifiers. Inventory changes preserve a ledger and supported unit conversions. These mechanisms make the calculation and transfer paths inspectable and give developers specific contracts to test.

The product remains a prototype because interface conflicts, an unidentified responder and policy disagreements limit integration readiness. Transaction code also needs concurrent database testing.

7.5 Limitations and threats to validity

Table 7.1 organises the principal threats by the inference they affect. Construct validity concerns whether a measure represents the intended outcome. Conditional simulated delivery does not represent completed delivery over all messages, and event displacement totals do not represent observed camp populations. Internal validity concerns the source of a comparison: thresholds were selected on the evaluated partition, child preprocessing uses held-out information, and population baselines differ in fitting window. External validity concerns transfer to actual links, camp cooking and unsurveyed age groups.

Table 7.1 Principal validity threats and required corrective evidence

Threat	**Affected inference**	**Corrective evidence**
Constructed state/action labels	Simulator score ≠ field reliability	Measured trace and complete outcomes
Threshold reuse / boundary overlap	Operating point ≠ independent test	Separate validation; purged final period

Threat	Affected inference	Corrective evidence
Child global preprocessing	Saved CV may reuse held-out information	Original files; fold-contained pipeline
Conditional delivery denominator	Deferred work omitted from success mean	All-message deadline-qualified acceptance
Different forecast windows	Gain cannot isolate robust estimator	Matched configurations and ablations
Survey transfer and missing upstream inputs	Derivative fit ≠ validated camp demand	Codebook rerun and independent camp records
Finite queues and partial fixtures	Persistence ≠ no-loss guarantee	Fault/restart/replay and expiry traces
Long horizon and uncalibrated ranges	Seven-day score ≠ 90-day validation	Horizon-specific error and range coverage
Incomplete UI, IDs and ownership	Prototype ≠ release-ready individual work	Build, isolation tests and contribution record

Data provenance limits are uneven. The local household derivative is available and reproducible, while the original survey conversion chain is incomplete. Child artefacts can be inspected but not refitted from original inputs. Generated population histories are fully identified as artificial, with unresolved event-export provenance. These differences require separate confidence in each result rather than treating all locally stored files as equally verified datasets. Earlier near-perfect formula-label fits are especially unsuitable as evidence for real demand accuracy.

Software validity is also bounded by fixtures. The current Pi failures concern expectations that disagree with the implemented priority mapping; the audit cannot determine the intended requirement solely from either side. Inventory tests omit a live database, and synchronization checks omit a complete broker/cloud fault trace. The finite buffer can lose or reject work, low/medium messages can expire, and retry exhaustion can leave records unresolved. A claim of universal exactly once application delivery or no message loss would therefore be unsupported.

Some implementation assumptions can affect the results beyond model error. Population envelopes are not calibrated for intervals, fixed cohort proportions can miss demographic shifts, and prefix-based roster replication or attendance selection can depend on record order. Long inventory horizons extend beyond the evaluated population horizons. The 3-litre cloud water basis also differs from local 15-litre distribution configuration. Each issue needs its own operational definition or

validation, since improving classifier accuracy cannot correct an incompatible quantity basis.

Candidate ownership, institutional records and operational validation remain draft-completion requirements in the missing-evidence register.

7.6 Implications

For development, the immediate implication is to align intended policy, source behaviour and test expectations before extending the models. The next priority is a traceable end-to-end sync experiment and a database-backed inventory trial. These steps address product uncertainty that an isolated model benchmark cannot resolve. Forecast evaluation should then use separate threshold validation and final periods, matched baselines and documented upstream preparation.

For camp planning, outputs should retain units, input dates, assumptions and an operator's ability to correct records. A forecast should inform a decision rather than conceal an unavailable measurement. For research, the prototype demonstrates why integrated systems need evaluation at both model and application boundaries. Once the missing operational data are collected under an appropriate protocol, the present contracts and evidence map can support a more credible assessment of practical benefit.

8 COMMERCIALISATION AND SUSTAINABILITY

8.1 Value proposition and target users

The proposed value is a shared, explainable view of camp population, consumable requirements, stock coverage, and pending synchronization. Likely operational roles include camp coordinators, storekeepers, and authorized relief-agency staff. These roles are inferred from the functions, not confirmed by customers. An operator could use a compatible stock outlook to plan replenishment while seeing whether recent local records have reached the cloud. Adoption depends on reliable recording, understandable units, and a workflow that fits the organization's responsibilities.

An initial pilot should use assisted planning with manual overrides, unavailable-result states and agreed operational responsibility. The proposed information benefit still requires measurement.

8.2 Market and competitive position

No verified market-size estimate, purchaser interview or commercial competitor trial was supplied. The comparison in Chapter 2 concerns capabilities of alternative techniques rather than current vendor pricing or market share. The prototype's potential distinction is its combination of local persistent operations, application-level message handling, and traceable camp-consumable calculations. Whether that combination is preferred over an existing inventory system or a simpler spreadsheet requires direct assessment by an adopting organization.

8.3 Business or sustainability model

A proposed sustainability model is institutional pilot funding followed by a support arrangement based on maintenance and operational responsibilities. The adopting organization would need to assign ownership of configuration, commodity aliases, data correction, backups, and model review. Local operation reduces some immediate dependence on the cloud but does not remove maintenance costs. Database, broker, hosting, connectivity and staff support remain part of total operating expenditure.

Table 8.1 identifies cost categories and the evidence required for a budget. No monetary values are fabricated. Hardware quotations need a specified bill of materials; hosting estimates need workload and retention assumptions; staff effort needs

observed support tasks. A useful pilot budget should also include training, replacement equipment, and data preparation. Sustainability depends as much on these recurring responsibilities as on initial software availability.

Table 8.1 Proposed operating model and budget evidence requirements

Responsibility / cost	**Proposed arrangement**	**Evidence required**
Hardware and replacements	Institution-owned pilot equipment	Specified bill of materials and quotations
Broker, database and hosting	Managed by authorised technical team	Workload, retention and provider budget
Connectivity and power	Camp/site operational responsibility	Observed link use and power requirements
Data preparation and aliases	Maintained with operational review	Staff effort and change-control record
Backups, support and training	Assigned support agreement	Recovery tests, task records and pilot feedback
Model maintenance	Periodic evidence-based review	Independent labels and accepted update criteria

8.4 Scalability intellectual property and compliance

Multi-camp operation should be gated by identity-isolation and load tests. Camp identifiers in payloads do not resolve a live map keyed only by local person ID. A larger queue limit also cannot be assumed to improve throughput without storage and recovery measurements. Scalability claims should follow tests with colliding IDs, concurrent inventory adjustments, outages, and varied message arrival rates. The current evidence establishes implemented mechanisms, with those deployment properties still unresolved.

Ownership of candidate, group and externally sourced work needs confirmation before licensing or commercial distribution. Dependency on licenses and dataset access conditions must be reviewed for the intended deployment and research sharing. This thesis makes no legal determination or claim of certified compliance. A release review should use the actual jurisdiction, institution and contracts, with authorized advice where required. Candidate contribution and historic assistance also need completion under M03 and M12.

8.5 Contribution to the Sustainable Development Goals

The component has a plausible functional alignment with Sustainable Development Goal 2, particularly access to sufficient food, and Goal 6, which concerns water and sanitation [20] [21]. Food planning and a clearly bounded drinking-water estimate can support information used in those areas. Functional alignment is not measuring impact: no change in hunger, equitable water access or humanitarian coverage was observed in this audit.

A future pilot should select outcomes within the system's actual reach, such as compatible stock reconciliation, timely replenishment decisions, and understanding of forecast uncertainty. Any SDG-related assessment would then connect those measured outcomes to the organization's programmed rather than equating a dashboard with achievement of a global target. Energy-saving and waste-reduction claims also need direct measurements before entering the sustainability case.

9 CONCLUSION AND FUTURE DIRECTIONS

9.1 Conclusion

This study examined the implemented Smart Resource Prediction and Network Prioritization component through source inspection, artefact reproduction, and selected software checks. The resource path combines damped population projection, explicit served-food rates, survey-derived raw-food coefficients, commodity matching, and inventory outlooks. The network path combines measured inputs, two saved random forests, application priorities, finite persistent queues, and cloud acceptance tracking. These mechanisms form a concrete prototype, with several integration and evidence limits still unresolved.

The numerical findings are mixed. Network argmax accuracy is 92.40%, below persistence at 93.57%, while the saved threshold policy detects 49.04% of generated escalations at a 12.28% false-alarm rate. Learned QoS increases conditional simulated delivery to 90.08% from 82.50%, alongside 12.23% buffering and no critical-delivery improvement. Damped population forecasting reduces seven-day generated scenario errors against the stated baselines. Food models explain limited individual or household variation, and independent camp-consumption validation remains unavailable.

9.2 Achievement of objectives

Table 9.1 relates to the documented architecture, reproduced experiments, and selected checks to the objectives. Operational validation remains partly achieved because real-link, database, stakeholder, and integration evidence is incomplete.

Table 9.1 Objective achievement within the available evidence

Objective	Achievement	Remaining work
O1	Architecture and contracts documented	Author ownership confirmation
O2	Saved network experiment reproduced	Independent measured final evaluation
O3	QoS and queue decisions analysed	All-message runtime-policy and fault trials
O4	Available fit/back-test reproduced; resource chain traced	Upstream rerun and camp validation

Objective	Achievement	Remaining work
O5	Selected checks executed and reported	Policy alignment, UI, live DB/broker tests

9.3 Individual research and product contributions

The supported contribution is a traceable account and evaluation of the component's joined planning and communication mechanisms. It documents current calculations, reproduces available results, and identifies specific weaknesses in their interpretation. The product contribution includes inspectable rate bases, persistent work handling, and explicit cloud acknowledgment contracts. Novelty of established algorithms and sole ownership of the wider group system are not claimed.

9.4 Future directions

The first development priorities are to reconcile priority requirements and tests, resolve the resource interface conflicts, verify or replace the legacy responder's contract, and run database-backed inventory and complete synchronization fault trials. Model work should then separate training, operating-point validation and final evaluation, purge future-label boundaries, and refit child preprocessing within folds when the original inputs become available. Matched window forecast ablations and empirical envelope coverage would strengthen the population's analysis.

Independent camp records and message traces should capture compatible quantities, input freshness, reconciliation, deferral and deadlines. Stakeholder tasks and device benchmarks can then establish practical usefulness. The present evidence supports an evaluated prototype and a specific validation agenda.

REFERENCES

[1] L. N. Van Wassenhove, “Humanitarian aid logistics: supply chain management in high gear,” Journal of the Operational Research Society, vol. 57, pp. 475–489, 2006, doi: 10.1057/palgrave.jors.2602125.

[2] SDRC-Box project, “SmartCamp-IoT-AI-Framework,” supplied source repository, working-tree snapshot associated with base commit 9b8bfd04828716be3f0aabfa75279f457b9502e8, branch intel-res-msg, inspected Oct. 9, 2026. Local research artefact; module paths and hashes are recorded in the accompanying evidence map.

[3] A. Banks and R. Gupta, Eds., “MQTT Version 3.1.1,” OASIS Standard, Oct. 29, 2014. [Online]. Available: https://docs.oasis-open.org/mqtt/mqtt/v3.1.1/os/mqtt-v3.1.1-os.html. Accessed: Oct. 9, 2026.

[4] Sphere Association, The Sphere Handbook: Humanitarian Charter and Minimum Standards in Humanitarian Response, 4th ed. Geneva, Switzerland: Sphere Association, 2018. [Online]. Available: https://spherestandards.org/wp-content/uploads/Sphere-Handbook-2018-EN.pdf. Accessed: Oct. 9, 2026.

[5] F. Palmese, A. E. C. Redondi, and M. Cesana, “Adaptive Quality of Service Control for MQTT-SN,” Sensors, vol. 22, no. 22, Art. no. 8852, 2022, doi: 10.3390/s22228852.

[6] J. Jeong, S. Park, and Y. Kyung, “Network-Aware QoS Optimization for MQTT Broker-Based Message Delivery,” Journal of Internet of Things and Convergence, vol. 11, no. 4, pp. 91–98, 2025, doi: 10.20465/KIOTS.2025.11.4.091.

[7] L. Breiman, “Random Forests,” Machine Learning, vol. 45, pp. 5–32, 2001, doi: 10.1023/A:1010933404324.

[8] National Institute of Standards and Technology, “THEIL-SEN FIT,” Dataplot Reference Manual. [Online]. Available: https://www.itl.nist.gov/div898/software/dataplot/refman1/auxillar/ts_fit.htm. Accessed: Oct. 9, 2026.

[9] E. S. Gardner, Jr., and E. McKenzie, “Forecasting Trends in Time Series,” Management Science, vol. 31, no. 10, pp. 1237–1246, 1985, doi: 10.1287/mnsc.31.10.1237.

[10] A. E. Hoerl and R. W. Kennard, “Ridge Regression: Biased Estimation for Nonorthogonal Problems,” Technometrics, vol. 12, no. 1, pp. 55–67, 1970, doi: 10.1080/00401706.1970.10488634.

[11] R. J. Hyndman and G. Athanasopoulos, Forecasting: Principles and Practice, 3rd ed. Melbourne, Australia: OTexts, 2021, sec. 5.10. [Online]. Available: https://otexts.com/fpp3/tscv.html. Accessed: Oct. 9, 2026.

[12] V. Cerf et al., “Delay-Tolerant Networking Architecture,” RFC 4838, Apr. 2007, doi: 10.17487/RFC4838.

[13] S. Blake, D. Black, M. Carlson, E. Davies, Z. Wang, and W. Weiss, “An Architecture for Differentiated Services,” RFC 2475, Dec. 1998, doi: 10.17487/RFC2475.

[14] B. Efron, “Bootstrap Methods: Another Look at the Jackknife,” The Annals of Statistics, vol. 7, no. 1, pp. 1–26, 1979, doi: 10.1214/aos/1176344552.

[15] Upfield Research and Development BV, “Dietary Intakes of Sri Lankan Children 6–14 years, 2021,” survey ID LKA_2021_DISLC_v01_M_v01_A_ESS, FAO Microdata Catalogue. [Online]. Available: https://microdata.fao.org/index.php/catalog/2963. Accessed: Oct. 9, 2026.

[16] International Food Policy Research Institute, “BRIGHT Integrated Household Survey of Sri Lanka, 2024–2025,” Harvard Dataverse, V1, 2026, doi: 10.7910/DVN/FH0DSN. Publisher description: https://www.ifpri.org/project/bright-sri-lanka/. Accessed: Oct. 9, 2026.

[17] A. R. Hevner, S. T. March, J. Park, and S. Ram, “Design Science in Information Systems Research,” MIS Quarterly, vol. 28, no. 1, pp. 75–105, 2004, doi: 10.2307/25148625.

[18] Scikit-learn developers, “Common pitfalls and recommended practices,” scikit-learn documentation. [Online]. Available: https://scikit-learn.org/stable/common_pitfalls.html. Accessed: Oct. 9, 2026.

[19] PostgreSQL Global Development Group, “Explicit Locking,” PostgreSQL Documentation, sec. 13.3. [Online]. Available: https://www.postgresql.org/docs/current/explicit-locking.html. Accessed: Oct. 9, 2026.

[20] United Nations Department of Economic and Social Affairs, “Goal 2,” Sustainable Development Goals. [Online]. Available: https://sdgs.un.org/goals/goal2. Accessed: Oct. 9, 2026.

[21] United Nations Department of Economic and Social Affairs, “Goal 6,” Sustainable Development Goals. [Online]. Available: https://sdgs.un.org/goals/goal6. Accessed: Oct. 9, 2026.

GLOSSARY

Application acknowledgment. A message identifying that the cloud processing path accepted a logical synchronisation envelope. It is separate from MQTT transport completion.

Cohort. A population group used in the demand calculation, such as an age band. Fixed proportions are a forecasting assumption.

Conditional simulated delivery. The average constructed probability for actions selected to transmit; buffered actions are outside its denominator.

Damped trend. An extrapolated slope whose increments shrink with horizon. In this service each increment is multiplied by a power of 0.8.

Escalation. A future network severity greater than the current severity. Warning recall need not identify the exact future class.

Heuristic envelope. A range formed by a rule, here based on residual spread. No verified nominal prediction-interval coverage is implied.

Idempotent processing. Handling repeated instances of the same logical request without repeating its intended effect, subject to the actual persistence contract.

Raw-food basis. Solid food mass used for stock and procurement calculations, distinct from cooked served-meal mass.

Rolling origin. A forecast evaluation that advances the information cutoff and scores future values separately by horizon.

Working-tree snapshot. The inspected files including current changes beyond the base Git commit. It is the relevant source state for this audit.

APPENDICES

Appendix A Research instruments or extended results

A.1 Extended network and population results

Table A.1 reports the saved argmax class summary, whose overall accuracy was independently reproduced. Table A.2 provides reproduced population MAE by horizon. Saved class details and rounded back-test values retain their original evaluation settings; these tables do not add field observations.

Table A.1 Saved argmax class metrics on the reproduced evaluation partition

State / support	Precision / recall (%)	F1 (%)
STABLE / 4645	97.13 / 99.22	98.17
DEGRADED / 2697	91.83 / 87.47	89.59
CRITICAL / 656	61.70 / 64.33	62.99

Table A.2 Reproduced population MAE in people across generated horizons

Horizon / pairs per scenario	Normal: damped / flat / OLS MAE	Burst: damped / flat / OLS MAE
1 / 3,600	307.6 / 360.3 / 401.6	363.4 / 404.5 / 551.3
2 / 3,400	515.2 / 731.8 / 753.2	605.4 / 804.4 / 1,065.1
3 / 3,200	738.7 / 1,117.7 / 1,156.1	842.9 / 1,197.2 / 1,621.6
4 / 3,000	991.5 / 1,515.2 / 1,602.3	1,064.6 / 1,579.5 / 2,229.1
5 / 2,800	1,258.0 / 1,921.6 / 2,085.8	1,284.3 / 1,963.3 / 2,908.8
6 / 2,600	1,538.1 / 2,328.4 / 2,589.7	1,530.8 / 2,347.7 / 3,641.0
7 / 2,400	1,835.2 / 2,748.3 / 3,096.1	1,763.4 / 2,744.8 / 4,405.5

A.2 Reproduction commands and test interpretation

The commands below document the available audit procedures. Run them from the project root or indicated backend directory in a compatible environment. Test outcomes in this thesis come from the retained logs; these commands are provided for repetition, not as a claim that a new execution occurred during document rendering.

Project root: python3 output/reproduce_network_metrics.py

Cloud backend: node tests/inventory.service.test.js

Cloud backend: node tests/resources/commodityStandardizer.test.js

Cloud backend: node tests/resources/stockRunout.test.js

Cloud backend: node tests/resources/prediction.test.js

The network reproduction regenerates the 40,000-sample requested trace without saving a replacement training CSV and scores the existing models. The household reproduction uses LinearRegression(fit_intercept=False), KFold(5, shuffle=True, random_state=0) and the six age-band columns of lka_household_food.csv. It does not recreate original BRIGHT modules or the child input CSVs.

The local suites require initialised isolated CAMP_DB_PATH, PAGES_DB_PATH and DISASTER_DB_PATH databases. The reportable log is raspberry_network_sync_tests_initialized.log. Its eleven disagreements cover check-in, operation distribution, person rows, distribution-row types and integration order. The earlier uninitialized log is retained solely to document fixture correction.

cloud_resource_tests_detailed.log contains 36 individual assertions. A separate file-level Node runner summary is not used as an assertion of total. network_model_tests.log records twelve passing Python checks. population_backtest.log contains the complete generated scenario error report, including undamped results and horizon bias. These logs should accompany any later revised experiment.

A.3 Selected API and persistence specification

Table A.3 expands selected write and snapshot contracts. Parameters and status fields must be checked against the inspected source when an integration client is changed. Local queues and cloud inventory are separate stores; a schema arrow in Figure 4.6 is not a cross-store foreign key.

Table A.3 Selected write, prediction-input and synchronisation contracts

Contract	Meaning	Verification boundary
POST /api/network/state	Recent state, metrics and four-priority policy	Monitor/controller interface; not cloud delivery
POST /api/resources/inventory/items/:id/adjust	Receipt/issue/correction; supported original unit and base quantity	Transaction source inspected; concurrency trial absent
GET /api/resources/inventory/prediction-dataset	Inventory, roster/households and population history	Inference input, not online model training
POST /api/resources/inventory/prediction-snapshots	Store structured forecast context	No independent live database trial
smartcamp/sync/<entity>	UUID sync_id and entity envelope	Cloud ACK marks logical acceptance
forecast_request / forecast_result	Legacy local prediction contract	Matching deployable responder not identified

Inventory items retain the current base-unit balance. Movements preserve the signed adjustment and original-unit context; prediction snapshots preserve JSON calculation context. Message-buffer rows record priority and pending/flushed status. Sync-queue entries retain identifiers, retry state and acknowledgment association. Exact SQL and current repository paths are mapped in the companion evidence file so this summary cannot be mistaken for a new migration.

A.4 Evidence identification and unavailable inputs

Table A.4 records selected SHA-256 values to identify the derivative and model artefacts used in the audit. Hashes identify files; they do not certify dataset correctness, safe deployment or individual ownership. Full inventory and template hashes are retained in repository_audit.json and template_audit.json.

Table A.4 Selected evidence-file identifiers

Artefact	SHA-256 (wrapped for legibility)	Role
lka_household_food.csv	dffee8f8abe88487ded775bfbc10c950c9f7128419c1f4394c99a6680cbba241	Derived-data fit
network_model.pkl	d036be1e5a186a052106f3b5f227e801d97dc81cd997782cdd7a4735d9a5d7f5	Saved state classifier
qos_model.pkl	7c0d9b951ec081782c41fe735f3ef2f01b45481bf40d7cbcfd0ecae8b84c5195	Saved QoS classifier

The missing-evidence register, M01–M27, identifies unavailable upstream inputs and operational observations. Saved quantile summaries and earlier formula-label notebooks remain historical artefacts outside the principal result claims.

APPENDIX B: ACADEMIC INTEGRITY AND AI USE DISCLOSURE

Table B.1: Generative AI use in this work

Item	Disclosure
Tool	Claude Code (Anthropic), codex, a coding and writing assistant.
Purpose	Writing and refactoring code.
Representative workflow	Described a requirement or a defect; the tool proposed code or text; the author ran the code, inspected the outputs and the data.
Sections and artefacts affected	Code under backend/physiological, StressApp and web
Responsibility	The author remains responsible for the accuracy, originality, citations, code, data, analyses and conclusions in this thesis.